University of Southampton

Faculty of Engineering and Physical Sciences
School of Electronics and Computer Science

Localisation of Autonomous Vehicle using LiDAR Sensor

by

Ketan Ingale

(September 2024)

Supervisor: Dr Daniel Clark
Second Examiner: Dr Luis-Daniel Ibáñez

A dissertation submitted in partial fulfilment of the degree of
MSc Data Science

Abstract

Autonomous vehicles are the future of transportation. The technology has been evolving rapidly, from simple assistance features like lane keeping, collision avoidance, etc., to current level 3 autonomy where the car drives itself to its destination. This dissertation helps investigate the localisation of autonomous vehicles using LiDAR sensors in environments where the GPS signal is unreliable or unavailable. The study is conducted using the KITTI-CARLA dataset. This dataset is designed to mimic real-world driving scenarios. Data include LiDAR points across varied urban, rural, and highway environments.

Some traditional methods such as K-Means clustering, Convex Hull, Minimum Enclosing Circle, and Iterative Closest Point (ICP) have been used for localisation. However, these methods are found to have some limitations. They frequently miss the environmental details that are necessary for accurate localisation. To solve this issue, an advanced mathematical model is developed, incorporating derivative analysis and dynamic parameter adjustments. This technique significantly enhances the accuracy of localisation by adapting to complicated data features. By adjusting to intricate data features, this method greatly improves localisation accuracy. It drastically lowers average localization errors across all evaluated scenarios and performs far better than the standard methods.

For example, in Town 01, the advanced model reduces errors by around 87% when compared to the Convex Hull and Minimum Circle approaches, and by about 95% when compared to the ICP method. The findings obtained in seven distinct towns, each with its unique environment, demonstrate the efficiency and robustness of the model. According to the research, the sophisticated mathematical model may greatly improve autonomous vehicle localization, and it works especially well in a variety of difficult settings. By providing a solid foundation for upcoming advancements and a dependable substitute for GPS-based systems, it seeks to advance the field of autonomous vehicle technology.

This dissertation is submitted for the MSc in Data Science at the University of Southampton. The work has been conducted under the supervision of Dr Daniel Clark, with Dr Luis-Daniel Ibáñez as the second examiner.

Statement of Originality

- I have read and understood the ECS Academic Integrity information and the University's Academic Integrity Guidance for Students.
- I am aware that failure to act in accordance with the Regulations Governing Academic Integrity may lead to the imposition of penalties which, for the most serious cases, may include termination of programme.
- I consent to the University copying and distributing any or all of my work in any form and using third parties (who may be based outside the EU/EEA) to verify whether my work contains plagiarised material, and for quality assurance purposes.

You must change the statements in the boxes if you do not agree with them.

We expect you to acknowledge all sources of information (e.g. ideas, algorithms, data) using citations. You must also put quotation marks around any sections of text that you have copied without paraphrasing. If any figures or tables have been taken or modified from another source, you must explain this in the caption and cite the original source.

I have acknowledged all sources, and identified any content taken from elsewhere.

If you have used any code (e.g. open-source code), reference designs, or similar resources that have been produced by anyone else, you must list them in the box below. In the report, you must explain what was used and how it relates to the work you have done.

I have not used any resources produced by anyone else.

You can consult with module teaching staff/demonstrators, but you should not show anyone else your work (this includes uploading your work to publicly-accessible repositories e.g. GitHub, unless expressly permitted by the module leader), or help them to do theirs. For individual assignments, we expect you to work on your own. For group assignments, we expect that you work only with your allocated group. You must get permission in writing from the module teaching staff before you seek outside assistance, e.g. a proofreading service, and declare it here.

I did all the work myself, or with my allocated group, and have not helped anyone else.

We expect that you have not fabricated, modified or distorted any data, evidence, references, experimental results, or other material used or presented in the report. You must clearly describe your experiments and how the results were obtained, and include all data, source code and/or designs (either in the report, or submitted as a separate file) so that your results could be reproduced.

The material in the report is genuine, and I have included all my data/code/designs.

We expect that you have not previously submitted any part of this work for another assessment. You must get permission in writing from the module teaching staff before re-using any of your previously submitted work for this assessment.

I have not submitted any part of this work for another assessment.

If your work involved research/studies (including surveys) on human participants, their cells or data, or on animals, you must have been granted ethical approval before the work was carried out, and any experiments must have followed these requirements. You must give details of this in the report, and list the ethical approval reference number(s) in the box below.

My work did not involve human participants, their cells or data, or animals.

ECS Statement of Originality Template, updated August 2018, Alex Weddell aiofficer@ecs.soton.ac.uk

Acknowledgement

First and foremost, I would like to convey my heartfelt thanks to my supervisor, Dr. Daniel Clark. His knowledge and perspective were extremely helpful in deciding the direction and path of this research project. His determination and commitment to excellence influenced every phase of this dissertation.

I am also extremely thankful to my second examiner, Dr. Luis-Daniel Ibáñez. His thoughtful criticisms and valuable feedback have greatly improved the standard of this project.

I offer my genuine gratitude to the Faculty of Engineering and Physical Sciences and the School of Electronics and Computer Science at the University of Southampton. The amenities and the encouraging academic atmosphere have enabled me to work on the dissertation successfully and find it fulfilling. I would also like to express my gratitude to the administrative and technical team members in the department. Their help in resolving technical problems and providing the required resources guaranteed the smooth progression of my research.

Additionally, I am thankful for the continuous support and motivation of my fellow classmates. Special thanks to my family and friends whose enduring support and understanding helped me stay focused and dedicated on my work, especially during the most challenging times. Their unwavering belief in my capabilities kept me inspired.

This dissertation is submitted in partial fulfilment of the requirements for the degree of MSc Data Science at the University of Southampton, in September 2024. It is a testament to the collaborative effort and scholarly environment provided by the university, for which I am deeply thankful.

Table of Contents

List of Figures

List of Abbreviations

ALV	Autonomous Land Vehicle
AV	Autonomous Vehicles
CARLA	Car Learning to Act
CPU	Central Processing Unit
DARPA	Defense Advanced Research Projects Agency
DR	Dead Reckoning
EKF-SLAM	Extended Kalman Filter – Simultaneous Localisation and Mapping
G-ICP	Generalised – Iterative Closest Point
GM	General Motors
GNSS	Global Navigation Satellite System
GPS	Global Positioning System
GPU	Graphics Processing Unit
HD	High Definition
ICP	Iterative Closest Point
IMU	Inertial Measurement Unit
INS	Inertial Navigation System
KITTI	Karlsruhe Institute of Technology and Toyota Technological Institute
LGPR	Localizing Ground-Penetrating Radar
LiDAR	Light Detection and Ranging
MEMS	Micro-Electro-Mechanical Systems
NDT	Normal Distributions Transform
NHOA	No Hands Across America
NLOS	No Line-Of-Sight
ODD	Operational Design Domain
OTA	Over the Air
PF	Particle Filter
PROMETHEUS	Programme for a European Traffic of Highest Efficiency and Unprecedented Safety
Radar	Radio Detection and Ranging
RANSAC	Random Sample Consensus
RFID	Radio Frequency Identification
RMSE	Root Mean Square Error
SAE	Society of Automotive Engineers
SLAM	Simultaneous Localisation and Mapping
Sonar	Sound Navigation and Ranging
SRI	Stanford Research Institute
TC	Tightly Coupled
V2V	Vehicle to Vehicle

Chapter 1

Introduction

CENTURIES ago, humans invented the wheel, which was one of the greatest inventions of all time. With this invention, began a new era in transportation, drastically shaping how we move across the landscapes and ultimately, how civilizations expanded and interacted. However, over the years, unprecedented advancements in automobiles and information technology have transformed the traditional vehicle from an old-fashioned source of commute to a full-fledged, smart, and infotainment-rich computing and commuting machine on the move [1].

Today, development in technology has led to the creation of an advanced form of vehicle referred as autonomous vehicles, driver less cars or certain other names which all pertain to intelligent vehicles. It represents a similar revolutionary step in redefining modern transportation by integrating advanced technology with everyday vehicle use. Autonomous vehicles are equipped with a variety of sophisticated sensors including cameras, radars, and LiDAR (Light Detection and Ranging), working in unison with the intelligent machine learning algorithms. These aim to make the commutes more efficient and safer, without the need for input from the humans. The algorithms in these vehicles are designed to optimize the driving patterns which help in reducing traffic congestion and lowering emissions; thereby contributing to a more sustainable urban environment.

New wireless transmission technologies, new embedded/sensor technologies, ad-hoc networks, data acquisition and analysis resulted in new vehicles and their modification [1]. Thus, there has been a shift toward higher levels of automation, transitioning from cruise control and lane-departure monitoring systems to autopilot systems designed to work autonomously. Automated vehicles are not just restricted to the advancement of technology it has also to deal with the legal, moral and social impact strategy of the AV.

1.1 Background

Through the collaboration of artificial intelligence, mechanical engineering, and advanced computing, automobiles were invented that could drive themselves. This development has changed the face of transportation as we know it. By enhancing safety, reducing pollution, and easing traffic, these cars have the potential to completely change the way we commute.

The Society of Automotive Engineers (SAE) has categorized autonomous technology into six levels of automation, from Level 0 (no automation) to Level 5 (full automation), with each level representing the extent to which a vehicle can take over driving tasks [7].

- **Level 0 (No Automation):** This level represents the traditional human driver where the driver of the vehicle has full control over all the aspects of driving without any automation.
- **Level 1 (Driver Assistance):** Even though the vehicle has a few automated capabilities, such as adaptive cruise control and lane-keeping assistance, the majority of driving activities still require the driver's entire attention and responsibility.

- **Level 2 (Partial Automation):** At this level, several automated processes – like steering and acceleration – operate simultaneously, but the driver must always stay alert, aware of their surroundings, and prepared to take over at any time.
- **Level 3 (Conditional Automation):** At this level, the car starts to take control of most of the systems. Without any assistance from humans, the car is capable of handling every aspect of driving in some situations. Despite this, the driver must be alert all the time in case the car requires the human to over the control. At this stage, the driver's duties start to gradually become more dependent on the autonomous systems in the car.
- **Level 4 (High Automation):** Under certain circumstances (the Operational Design Domain, or ODD), vehicles are capable of operating independently without requiring any kind of human intervention. These cars might still allow for some human operation under out-of-ODD situations.
- **Level 5 (Full Automation):** At this ultimate level, there is no need for human intervention at any time or place, indicating complete autonomy. The car can handle every driving task in every situation.

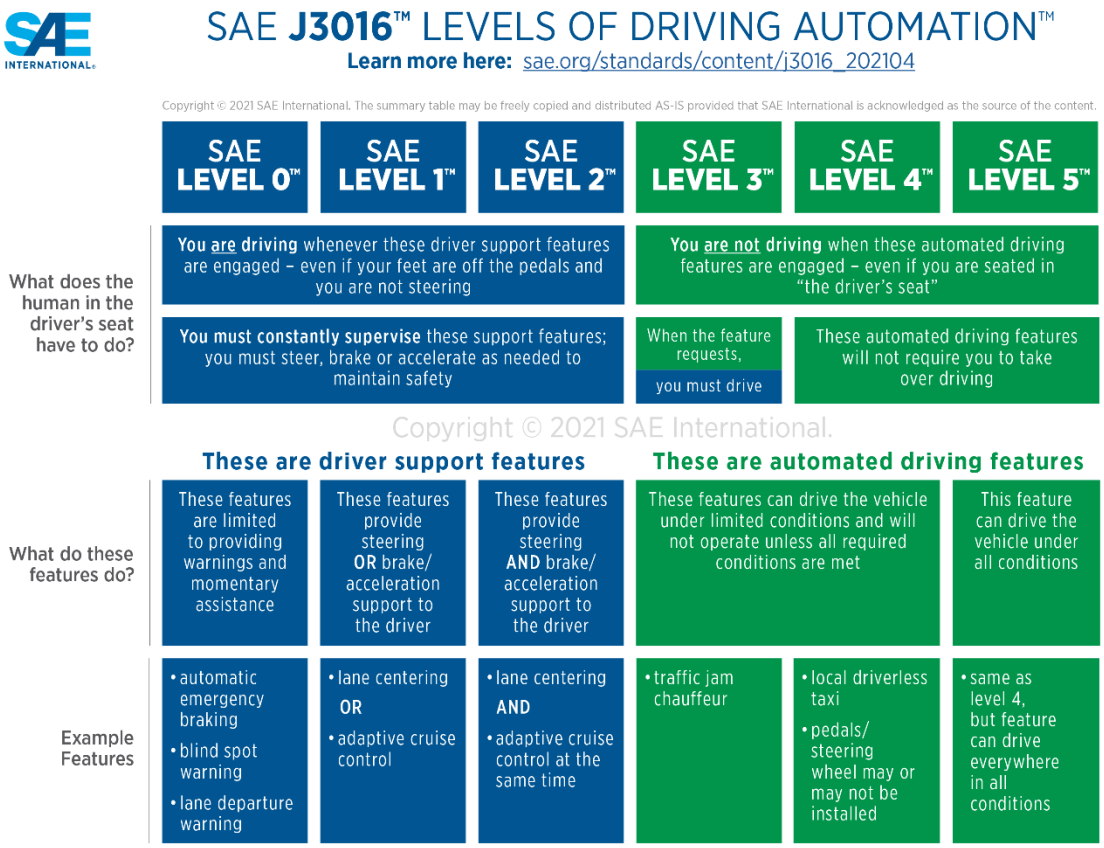

Figure 1: *SAE J3016 Levels of Driving Automation (Source: SAE International) [7]*

The change from these levels to redundancy in the scenario of complete automation emphasizes how the human driver's function evolves from active participant to passive spectator and finally to redundancy. As sensor technology, artificial intelligence, and machine learning progress, so does automation, enabling cars to perceive and understand their environment more accurately.

The sensors in autonomous vehicles are the essential parts of the system. Autonomous cars can incorporate several sensors, such as LiDAR, Radar, Sonar, GPS, IMU, and wheel odometry as seen in figure 2. These sensors are used to collect information about the surroundings which is then processed by the computer to regulate the steering, braking, and speed of the vehicle. Along with the automotive sensors, the information from environmental maps which is stored in the cloud and the data uploaded from the other cars are also used to make decisions about vehicle control [8].

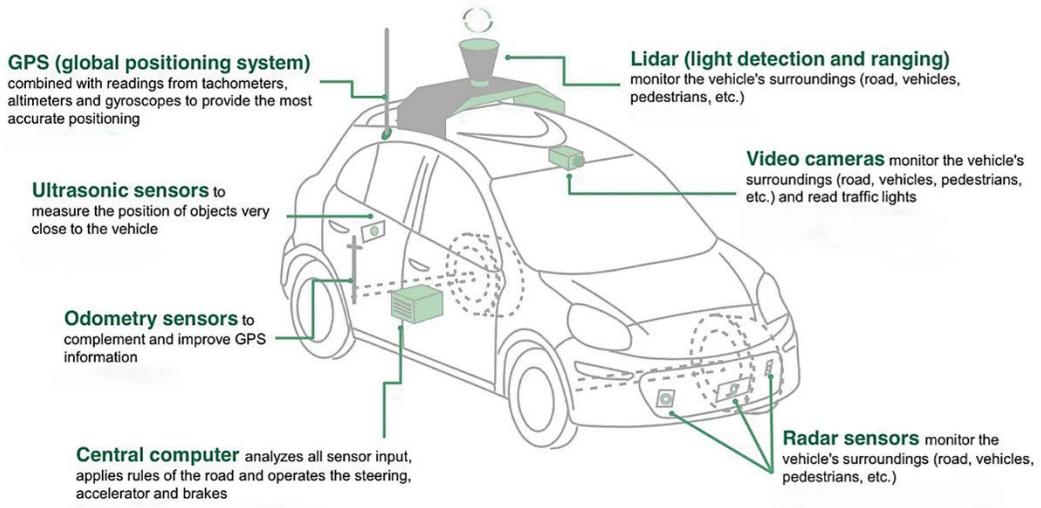

Figure 2: *Sensors on Autonomous Cars (Source: Wired) [9]*

1.1.1 Camera

Autonomous vehicles' perception systems rely heavily on cameras since they provide a different kind of data and a level of detail compared to other sensor technologies like radar and LiDAR. Cameras are among the first sensor technologies used in autonomous cars; they offer vital visual inputs that let the cars "see" their surroundings. Complex image processing activities including lane marker detection, traffic sign recognition, and pedestrian and vehicle monitoring depend heavily on this capacity. These days, autonomous cars frequently use several cameras – up to twelve at times – that are positioned thoughtfully throughout the vehicle to provide a complete 360-degree picture.

Cameras are beneficial, but they also produce enormous amounts of data that need a lot of processing power to process in real-time. Because high-definition cameras may capture millions of pixels per frame at high frame rates, handling this data effectively requires sophisticated processing techniques and robust hardware. In addition to navigation, cameras are useful in applications like semantic segmentation and end-to-end driving systems, which completely automate driving [10][11]. Internal cameras also keep an eye on drivers' attentiveness and help them communicate with the car, therefore maintaining safety through constant supervision [12].

1.1.2 Radar

Radio Detection and Ranging (radar) is an important sensor utilized in most modern-day self-driving cars for various tasks. These consist of adaptive cruise control, active collision avoidance, blind spot monitoring, and collision avoidance. Though radar is already an established technology, it is being improved continuously for application in autonomous vehicles [13]. Its basic operating principle: the Doppler effect, even allows for the direct computation of velocity by the detection of frequency shifts in the reflected radio waves. This crucial speed information is beneficial to the sensor fusion algorithms for fast convergence.

Automotive radar is broadly divided into two types: short/medium range and long range. long-range radar, which can typically operate at 77 GHz, provides speed measurements and makes large-scale object detection up to 200 m and under various conditions, albeit at much lower resolution. Short– and medium-range radar, which operate in 24 GHz and 76GHz bands, respectively, are more affordable in low and intermediate ranges and can accurately measure distance and velocity. But its application is limited by longer wavelengths and broader beams which cause scattering and loss of the signal.

While being tasked with performance related to weather, radar does give advantages over cameras and LiDAR. It is more efficient in penetrating under vehicles which leads to better environmental mapping and object recognition. In cases where high-resolution images are required, radar can be disadvantageous because it is less data efficient and has much poorer angular accuracy than LiDAR even though it is less computing intensive. This strategic layering of LiDAR and cameras with radar sensors enhances the safety of the autonomous vehicle detour systems as it creates a multi-dimensional sensory platform.

1.1.3 LiDAR

Self-driving cars depend on a variety of sensor networks for the environment's accurate measurement and understanding. Out of these, LiDAR (Light Detection and Ranging) is considered one of the major ones. LiDAR sensors utilize an infrared beam of light, usually in the 900 nm range, to send a laser pulse towards a target and measure the time taken for the pulse to return after reflecting off the target in an object. There is a variation in the type of wavelengths certain LiDAR systems use to enhance operation under adverse conditions such as rain, fog, or other low visibility scenarios.

On the other hand, LiDAR sensors do not rotate about to measure the radial velocity of an object like how radar instils in most instances. They then have to derive the motion from the changes in the position of the targets that were sampled within the period of scanning. LiDAR is also able to use highly focused beams of laser to achieve even higher spatial resolution by covering every area vertically layer by layer and providing the surrounding region in the 3D point cloud. This kind of resolution is advantageous in mapping static environments, identifying objects and tracking moving features such as animals, people, and cars.

MEMS (Micro-Electro-Mechanical Systems) based systems are the new focus of LiDAR technology today that employ rotating micromirrors to aim laser beams whenever required. Just like radar’s phased array technology, this particular technique gets rid of the mechanical drawback of moving parts by electronically guiding the laser beam. Furthermore, coherent LiDAR systems have a huge benefit under moving conditions as the systems can determine an object’s speed accurately by examining the frequency changes of the light returned. LIDAR technology has many advantages as it has made accuracy possible but its use might be limited due to weather interference and the sensitivity of dirt on the surface of the sensor.

1.2 Localization

In the realm of self-driving vehicle technology, we can find the term “localization”, which involves the ability of a vehicle to determine its position with its surroundings. This is one such ability which is executively important for efficient navigation as well as management of the operations of the car where the internal car systems ascertain the exact location of the vehicle in a relative space. The system of Localization continuously determines the position and orientation of the vehicle in a 4D space performed with distinct and high accuracy by exploiting complex algorithms that combine data received using various sensors, such as GPS, INS and High-Definition Maps.

Global Positioning System: The purpose of GPS is to supply space-time data for self-driving cars. It is made up of three components: GPS receivers fitted onto the vehicles, ground control and monitoring stations and satellites circling the Earth [14]. Each satellite is equipped with a synchronous atomic clock and it uses pseudo-code to continuously broadcast navigational data. GPS satellites are placed in orbit such that at any given point in time, at least four satellites are available for users anywhere in the world [15]. To get the almost precise location, at least three GPS satellites should be connected to the receiver to solve the three positional dimensions and time, the concept also known as trilateration. When the GPS receiver receives the data from more than three satellites, it can compute its current position (latitude and longitude) and other parameters. Using this data, the vehicles can locate, navigate and carry out other GPS-related functions.

Inertial Navigation System: An INS is an electrical instrument that applies one or more ambient sensors to measure or characteristics change of an object's present position. INS incorporates different sensors of which the most basic two are accelerometers and gyroscopes. Since the movable objects can be in three-dimensional space covering the x, y and z coordinates, there are usually three accelerometers and three gyroscopes that measure the linear motion as well as its orientation. Other sensors often integrated with INS are GNSS receivers, magnetometers, and pressure sensors [16]. Through the use of IMU (a combination of accelerometers, gyroscopes, and sometimes magnetometers and pressure sensors [17]), INS enables the precise detection of high-precision 3D location, velocity and altitude information about the vehicles. It can compensate for the inadequate real-time GPS localization. For instance, the automobile can use INS to determine its location if the GPS signal is weak. In other words, self-driving automobiles can achieve precise and sufficiently real-time location updates thanks to GPS and INS [14].

High-Definition Maps: HD maps come in two map levels: static and dynamic, with a mind-blowing accuracy of 10 to 20 cm. To provide lane-level navigation, static HD maps—which are typically gathered beforehand—contain lane models (such as lane line, and slope), road information (such as traffic signs, zebra crossings, and direction signs), road attributes (such as statuses of road construction), and additional localization layers. Cloud-based dynamic high-definition maps contain real-time traffic data, including states of traffic congestion, accident reports, and traffic control data. Moreover, real-time updates to dynamic HD maps can be obtained from data supplied by infrastructures or other vehicles.

Precise localization is essential to the operation of autonomous cars. It has a direct influence on safe and efficient navigation. It guarantees that cars may navigate effective routes between locations while averting possible dangers or unlawful actions. For safe driving to minimize accidents resulting from lane drifting or unpredictable movements, the exact position of the vehicle helps to maintain consistent lane positions. With multi-lane highways and crowded city crossings, even little deviations can result in dangerous crashes. This accuracy facilitates interaction between the self-driving cars and the environment. They can detect and respond to moving and non-moving objects, such as cyclists, pedestrians, vehicles, and other unexpected objects like construction and trash. With localization, a vehicle can change its direction in an instant to avoid obstacles such as potholes or other unforeseen boundaries like traffic jams. This also ensures the different rules of the road such as direction changes, speed limits and many more are reciprocated which enhances safety as well as controls.

The goal of this research is to enhance autonomous cars' positional determination capabilities, particularly in the areas where the GPS signal quality is poor or non-existent, by utilizing LiDAR data. LiDAR captures high-resolution, three-dimensional images of the surrounding areas, which is essential because, unlike other sensors that may struggle in such circumstances, it is not dependent on external light sources and is less impacted by bad weather. LiDAR's precise and comprehensive data allows autonomous vehicles to triangulate the location precisely, improving navigation where GPS signals are weak or blocked by obstacles such as dense foliage, urban canyons, or tunnels. To take advantage of these benefits, the study creates sophisticated algorithms intended to efficiently integrate and process LiDAR data. The final objective is to create a localization framework that guarantees the safe operation of autonomous vehicles and reduces the possibility of mistakes that could result in navigational accidents. This will improve the overall dependability and safety of autonomous transportation systems in a range of operational scenarios.

1.3 Research Objectives

The main objective of this particular work is to design and test a LiDAR localization system which can be integrated into an autonomous vehicle in locations having no or poor GPS signal. In terms of the limits of this technology, using its spatial data provided by LiDAR, the system enhances the GPS recovery ability of the vehicle. This becomes more critical in circumstances where the standard GPS systems are ineffective such as highly dense urban areas, dense woods or tunnels. Successful implementation of this system seeks to eliminate several existing navigation as well as safety challenges posed by the autonomous automobile.

For a system to maintain course accuracy and prevent navigational hazards, improving the efficiency of the algorithms is necessary. This would ensure that the vehicle's location can be determined with the least amount of error. To accomplish this, real-time data filtering, analysis, and interpretation require the application of complex data processing techniques. The effectiveness and dependability of the localization system during regular operations are directly impacted by the precision of these algorithms.

Effectiveness remains another area of vital importance. The project aims to enhance the capacity of the system for efficient data management. Data has to be interpreted quickly if the car is to be able to make rapid decisions. It includes decreasing the computation burden on the car's onboard systems, gathering and analysing information more quickly, and improving the data flow. Increasing efficiency improves the safety and performance capacity of the car by providing quick responses to the environment.

1.4 Structure of Report

This dissertation is structured into six principal chapters. Every chapter aims to offer a thorough examination of different aspects of the study and its conclusions.

Chapter 1: Introduction – This chapter discusses the evolution of autonomous vehicle technologies. It also touches upon the importance of localization in autonomous vehicles. This chapter, in addition, delineates the research problem, the objectives and the scope of the study.

Chapter 2: Literature Review – In this specific part of the thesis, the previous literature regarding localization techniques in autonomous vehicles is critically evaluated. It examines previous issues and resolutions and concentrates on GPS and LiDAR studies.

Chapter 3: Methodology – This chapter details the methods used for collecting and analysing the data. It describes the experiment's setup, the collection of LiDAR data, and the algorithms for localization.

Chapter 4: Implementation – The implementation chapter focuses on the design and deployment of the localisation system. It focuses on the different hardware platforms, programming languages, and software packages used. This chapter highlights the practical challenges faced and the measures taken to counteract them.

Chapter 5: Results and Discussion – This chapter describes the results obtained during the study. The key focus on results is how well the developed mathematical model works as compared to the basic methodologies. The results are interpreted taking the errors statistics and accuracy into account. In the end, the usefulness of the system for future research on autonomous vehicle navigation is discussed.

Chapter 6: Conclusion and Future Work – The last chapter of the dissertation outlines the main findings and contributions. This chapter discusses the broader implications of the research and potential impacts on self-driving cars. It also indicates paths for future research, proposing methods to enhance localization technologies based on the results of this study.

Chapter 2

Literature Review

LIDAR-BASED localization has been the subject of research for a while now since it is more accurate and dependable that other sensors. This chapter summarizes the related work in the field. It assesses history of the autonomous vehicles along with the different localization techniques used. We also touch upon sensor fusion and the challenges faced in the navigation of the autonomous vehicles.

2.1 Historical Development of Autonomous Vehicles

The development of autonomous vehicles has been significantly influenced by early historical experiments. In 1926, the Houdina Radio Control demonstrated the initial attempt at autonomous vehicles was the radio-controlled car, called Linriccan Wonder. This radio-controlled car was a modified 1926 Chandler that had antennae on its rear compartment. This was operated by another car that was sending the radio impulses while following it, and the antennae caught the signals. These signals were sent to circuit-breakers which operated small electric motors that directed the car's movements. In late 1926, a modified version of Linriccan Wonder was showcased by the name "Phantom Auto". A few years later in 1939, GM (General Motors) had an exhibit named Futurama by Normal Bel Geddes's at the World Fair. This exhibit showcased embedded-circuit-powered electric cars where the circuits were embedded in the roadway and controlled by radio [18].

RCA Labs took this idea further in 1953. They built a miniature car in 1953. The wires were laid in the pattern on a laboratory floor and the car was controlled and guided by these wires. In 1958, L. Hancock and L. N. Ress scaled the idea of RCA Labs and experimented with it on the actual highway. General Motors collaborated with this experiment. In the 1960s, at the Auto Show "Motorama" (GM), General Motors showcased the Firebird. These were a series of experimental cars that held an electronic guide system. These cars could rush in a highway without driver's involvement [19][20][21]. In 1966, the Ohio State University started developing driverless cars that were powered by electric circuits buried in the road. At the same time, a Citroen DS autonomous car was tested by the Transport and Road Research Laboratory in the UK. This car interacted with the magnetic cables placed inside the road. This experiment proved to be successful. It travelled the test track at 130 kmph while maintaining constant speed and direction in a variety of weather conditions [22][23]. In the 1980s, Ernst Dickmanns and his colleagues at the Bundeswehr University Munich in Munich, Germany developed a robotic Mercedes-Benz vehicle with a visual guidance system. This van could travel up to speeds of 63 kmph on empty streets. This decade saw a lot of important and noteworthy, national and international projects in this field. One such project was the PROMETHEUS Project which was funded by Eureka. With more than 1 billion US dollars being invested in this project, the project lasted from 1987 to 1995 [3][4][24][25].

The U.S. Department of Defence's DARPA has been instrumental in the advancement of autonomous vehicle technology. Universities like Carnegie Mellon University, Environmental Research Institute of Michigan, University of Maryland, Martin Marietta, and SRI International were among the notable ones that contributed to the Autonomous Land Vehicle (ALV) Project. By completing the first road-following demonstration using computer vision, LIDAR, and autonomous control, this research achieved a noteworthy milestone. A robotic car was shown driving at up to 31 km/h in the demonstration [26][27].

Through its "No Hands Across America" (NHOA) expedition, Carnegie Mellon University's NavLab project made a major advancement in autonomous vehicle technology in 1995. During a 5,000-kilometre cross-country journey, the car attained 98.2% autonomous driving. The car was semi-autonomous; human operators controlled the brakes and throttle, while neural networks controlled the steering wheel [28][29]. Alberto Broggi of the University of Parma presented the ground-breaking Argo autonomous vehicle project in 1996. As part of this project, a Lancia Thema was modified to recognize and follow lane markers on traditional roadways on its own. The highlight of the attempt was when the modified car travelled 1,900 km at an average speed of 90 km/h over six days through northern Italy. Remarkably, the vehicle operated exclusively in automatic mode for 94% of the journey. The maximum continuous distance covered by the vehicle in automatic mode was 55 km. The car achieved this with just two inexpensive video cameras for navigation and employed stereoscopic vision algorithms to perceive its environment [30].

In the early 2000s, many countries began introducing autonomous public transportation. It was at this time that the Netherlands introduced the ParkShuttle, an autonomous public road transportation system [31][32]. Simultaneously, the US government started initiatives centred mostly on using autonomous vehicles for military purposes. These included DARPA's Demo II, the US Army's Demo I, and Demo III, all of which were heavily funded by the US government [33]. Demo III demonstrated how autonomous ground vehicles could navigate difficult off-road environments while deftly dodging hazards like trees and boulders. A Real-Time Control System was developed by NIST to assist these efforts. In addition to controlling individual vehicle functions like the throttle, steering, and brakes, this hierarchical control system arranged the motions of groups of vehicles to accomplish higher-level goals [34][35][36].

The following decade saw a huge improvement in autonomous driving. Companies like Tesla, Audi, and Volvo started releasing their versions of autonomous cars. These cars are now available for the general public to purchase with continuous improvements being made in the algorithms and deploying them to the cars via OTA updates. However, these cars are still considered to be SAE Level 3. By 2035, it is predicted that most of the cars will be fully autonomous and require no human control. The future awaits and is not distant [37].

2.2 Localization Techniques in Autonomous Vehicles

Self-Localisation Techniques of Autonomous Vehicles	
Sensor-based Techniques	
Active Sensors	**Passive Sensors**
- LiDAR-based Localisation	- GPS-based Localisation
- Radar-based Localisation	- IMU-based Localisation
- Ultrasonic-based Localisation	- Vision-based Localisation

Figure 3: *Different Sensor-based Localisation Techniques in Autonomous Vehicles*

A high-precision and reliable localization system is necessary for an autonomous vehicle to operate safely and efficiently. Figure 3 shows different techniques used in autonomous vehicles to find the location of the vehicle – GPS, LiDAR, Radar, etc. Each localization technique has its advantages and disadvantages. For example, GPS has wide coverage but is unreliable in places where signals are blocked. LiDAR offers high-resolution environmental data. This data is essential for object recognition and navigation. However, LiDAR sensors are very expensive and often unreliable in bad weather. Radar, though cheap and more reliable in bad weather, provides less details than LiDAR. Rich visual data from cameras is quite reliant on lighting conditions, yet it is very useful for applications like lane detection. This section

evaluates the research done on each localization technique and identify their advantages and disadvantages.

2.2.1 Active Sensor-Based Localisation

A. Lidar-Based Localisation

Conventional LiDAR localisation techniques, such as Iterative Closest Point (ICP) [2], Generalised-ICP (G-ICP) [3], and Normal Distributions Transform (NDT) [4]. These methods generally use geometric constraints to estimate vehicle motion. To improve their vehicle's localisation accuracy, K. Yoneda and S. Mita [5] used ICP to match real-time point clouds with previously recorded maps. An open-source framework called "Autoware" was created by S. Kato et al. [89]. This framework handled several self-driving modules, including those that use localisation methods like NDT and ICP. These methods, however, required precise initial location and would also struggle in situations which were feature-sparse, like highways.

To overcome these drawbacks, more recent methods restrict the solution space to three dimensions (x, y, and yaw) and incorporate LiDAR intensity data for extra ambient texture – like lane markings – that is generally not possible with only geometry data. Pre-built reference maps are often necessary for LiDAR-based localisation to match with point cloud data. If these maps are not available, real-time maps are created using SLAM techniques. This increases the position estimation accuracy but decreases the processing speed and memory efficiency [37][38]. New ideas like 1D corner maps make use of the vertical corner features of the city structures [39]. This method improves the positioning while requiring less data storage, but would struggle in the non-urban settings like grasslands, villages, etc. where there are no such structures. Some other techniques like 2D Occupancy Grid Map [40] incorporate road marking features to improve accuracy but raise data needs.

To improve localization accuracy and robustness in urban environments, Levinson et al. and further research have concentrated on 2D planar map matching using probabilistic mapping and SLAM-style algorithms [41][42]. Because of the sophisticated mapping and matching systems, the file size reaches up to 10MB per mile. [41][42]. This leads to a drastic increase in the amount of data storage needed. Alternative methods try to handle the complicated ground characteristics like snow-covered roads, while using less memory by use of curve approximation algorithms or road surface reflective intensity mapping [43][44][45].

By using environmental height data, 3D mapping algorithms improve localization precision, although this needs significant processing power [46]. Methods such as the 3D occupancy grid map show encouraging results in minimizing data size while preserving dependable location accuracy [47]. Future studies will focus on optimizing these factors as the continual advancement of lidar-based localization highlights a constant trade-off between accuracy, memory demands, and real-time processing capabilities.

B. Radar-Based Localisation

As compared to LiDAR-based and vision-based localisation techniques, radar-based localisation is preferred for its real-time performance. It also required less memory and processing power as compared to other techniques [48][49]. However, radar-based SLAM lacks detailed features. Because of this, it may experience problems with data registration during map matching, which could result in low positioning accuracy [50]. The Fourier-Mellin transform is used in techniques such as trajectory-oriented extended Kalman filter (EKF)-SLAM. This helps to progressively register radar pictures and calculate vehicle position without direct feature mapping. However, this approach can lead to a mean positioning inaccuracy of up to 13 metres [48]. On the other hand, attempts to increase robustness in dynamic situations have been successful. For example, a semi-Markov chain with a Levy process has been extended, and 83% of location estimates are within 0.2 meters [51]. Additionally, reference maps for unfavourable weather conditions have been created to increase dependability [52].

The density-based stream clustering method used by the cluster-SLAM technique manages radar data in dynamic situations, processing them efficiently with a compact 200KB map. Particle filter (PF) and this method together minimize measurement noise during environment scanning, which allows for accurate vehicle position calculation [53]. Further advancements in positioning speed include a joint spatial and Doppler-based optimization framework that achieves a positioning refresh rate of 17 Hz. This utilizes a sparse Gaussian mixture model for reference scans, thus significantly lowering computational complexity [54][55].

High accuracy and real-time performance are possible with advanced applications, such as the under-vehicle-mounted localizing ground-penetrating radar (LGPR) system. This device achieves a refresh rate of about 126Hz with a root mean square error (RMSE) of 12.7cm by scanning the road subsurface and eliminating weather-related signal interference. Despite its efficacy, additional adjustments are required to lower the LGPR array's height so that more passenger cars can use it [56]. The aforementioned developments underscore the continuous endeavours to enhance radar-based localization methods. Thus, striking a balance between precision and the requirement for effective data processing in various operational settings.

C. Ultrasonic-Based Localisation

Due to the low cost of ultrasonic sensors, ultrasonic-based localization is widely used in indoor robot applications. But its limited detection range and susceptibility to dust, humidity, and temperature, make its use limited in autonomous vehicle positioning. [57][58]. In their work, Moussa et al. [59] created an ultrasonic navigation system based on the Extended Kalman Filter (EKF) algorithm. In the absence of GPS, ultrasonic sensors are used as the main locator. With a refresh rate of roughly 92 Hz, this configuration reduces vehicle drift and improves system reliability. Its major position inaccuracy, which can go up to 7.11 meters, is present.

To determine the exact location of the vehicle, Jung et al. [60] combined ultrasonic sensors with encoders, gyroscopes, and digital magnetic compasses. They did this by employing SLAM. This approach takes a long time to update positions—averaging 10.65 seconds—and tends to collect Inertial Measurement Unit (IMU) inaccuracies throughout the lengthy SLAM calculations. As a result, the device can only keep its position correctly for roughly 5.2 meters when driving.

In conclusion, although low-cost and low-power, ultrasonic-based localization systems are neither accurate nor robust enough for autonomous driving. While the technology shows promise for some applications, it still has to be improved to meet the strict demands of AV positioning.

2.2.2 Passive Sensor-Based Localisation

A. GPS-Based Localisation

For autonomous vehicles (AVs), GPS provides an economical and effective solution for localisation. However, it faces challenges from non-line-of-sight (NLOS) situations, multipath effects, and signal obstructions in urban canyons [61]. Position correction methods are one of the latest developments in GPS-based localization approaches to address these problems. To improve accuracy and dependability, these techniques combine many data sources, apply aberrant signal filtering, and make use of map aids [62][63]. For example, a unique method outlined in reference [64] combines information from GPS, RFID, and vehicle-to-vehicle (V2V) communications to improve GPS-based localization. Rather than relying just on connections that satisfy specific accuracy requirements, this approach analyses the accuracy of many data sources to increase resilience in GPS-degraded settings. With a computational complexity that is roughly 0.8% lower than earlier systems, the resultant system achieves a location accuracy of roughly 2.9 meters [65].

Innovative methods to improve GPS accuracy have been developed, including signal identification that adjusts outputs according to signal quality [66]. Lu et al. [67] tried to improve GPS accuracy by utilizing an open-source, low-precision map. However, they had difficulties while attempting to extract lane

markings at intersections. GNSS-based techniques improve localisation by eliminating abnormal GPS signals and adding terrain height from digital maps [68]. GNSS accuracy is also intended to be improved by efforts to account for NLOS signal delays. Even with these improvements, location RMS errors in urban canyons remain about 10m, making it difficult to achieve high precision.

B. IMU-Based Localisation

Strong anti-interference capabilities enable the IMU to measure acceleration and pitch rate effectively [69]. IMUs, however, are unable to independently determine precise positions over extended distances because of the accumulation of inaccuracies. When primary sensors momentarily fail, they are frequently utilized in conjunction with secondary sensors to guarantee continued localization [70]. Many approaches have been developed to deal with these issues. To improve accuracy in urban canyons, for instance, a dead reckoning (DR)-based tightly coupled (TC) method was presented [71]. Similarly, a modified TC strategy with aberrant GPS measurement rejection was employed to guarantee continuous location in situations where GPS is denied [72].

Additional developments include the occupancy grid-constrained prediction models developed by Wang et al. [73] to reduce GPS multipath interference and cumulative DR mistakes. To increase cost-effectiveness, Xiao et al. [74] used a particle filtering (PF) smoothing technique that took non-Gaussian noise into account. However, this required improvements in real-time performance. Belhajem et al. [75], in contrast, only achieved meter-level accuracy when using machine learning to rectify IMU aberrations during GPS outages.

Ndjeng et al. [76] and Gruyer et al. [77] proposed DR-based techniques that improve location integrity and accuracy, particularly in GPS-compromised contexts. They achieved this by employing restricted probabilistic models and the Transferable Belief Model (TBM) to boost localization robustness. Another method involves matching vehicle motion patterns against a pre-built map using pattern recognition techniques on IMU-generated pitch rate signals to calculate vehicle position. This method achieves reasonable accuracy without cumulative errors, but it is susceptible to measurement noise [69][78][79].

C. Vision-Based Localisation

New developments in multi-core CPUs and GPUs that enable heavy parallel image processing are driving the growth of vision-based localization [80][81]. One approach was created to identify symmetric park markings for autonomous parking. This was done using four fisheye cameras and a pre-made map. The method achieved a parallel position error of only 0.3 metres and a positioning time of 0.04 seconds [82]. Du and colleagues [83] further refined the method by applying a sequential RANSAC algorithm to effectively extract lane lines from photos. This method yielded a refresh rate of 0.12s and a location error of approximately 0.06m. Another approach was building a lightweight 3D semantic map for markers on the road. This strategy greatly decreased memory consumption and increased matching performance. Although it still needs more testing on curved roads [84].

To enhance vision-based localisation even further, planar feature maps including many attributes like texture, location, and orientation should be constructed. Refresh rates of roughly 0.09 seconds could be achieved by combining these characteristics with live camera views [6]. To update vehicle-pose probability distributions, high accuracy and real-time performance improvements were also obtained by matching camera images with predetermined orthogonal maps. This resulted in a positioning accuracy of less than 0.14m (RMS) and a performance time of roughly 0.057s [85][86]. These methods are mostly limited as they heavily rely on information from the visible road surface.

Alternative methods concentrate on merging classic image-matching algorithms with semantic descriptions. This helps increase the resilience of localisation in varying environmental circumstances. An example of this is the topological model that correlates characteristics from reference map nodes near collected photos. This results in a positioning accuracy of 0.45m [87]. However, sensitivity to lighting conditions presented difficulties. Another method achieved a confidence level of around 85.5% but with a slower refresh duration of 2s. This method involved utilizing an extended hull census transform for

semantic feature extraction from omnidirectional pictures [88]. These advancements demonstrate the continuous work to improve vision-based localization techniques for more precise and dependable autonomous car navigation.

Chapter 3

Methodology

THIS chapter outlines the methods used in this study to improve the LiDAR-based autonomous vehicle localisation methods where the GPS signal is low or unavailable. This section will cover the characteristics, preprocessing methods and the origins of the dataset. These fundamental aspects have a big influence on how well the localisation algorithms work. The main focus of the chapter is algorithm design. It includes an explanation of the data handling strategies, computational procedures, and theoretical foundations required for accurate localisation. The approach used ensures a thorough examination of the algorithms' efficiency and also provides a clear path from research questions to conclusions drawn in later parts.

3.1 Dataset

The dataset used in this research is the KITTI-CARLA dataset [90]. This dataset was created using the CARLA v0.9.10 which is an open-source simulator that enables the simulation of several sensors, more importantly, cameras and LiDARs [91]. This dataset is designed to replicate the sensor setup of the widely recognised KITTI dataset [92][93]. The car is mounted with a LiDAR, placed on top of the car, exactly at the centre of the roof. It has the same specifications as the Velodyne HDL-64E Lidar featuring 64 channels with a vertical field-of-view from -24.8° to +2°, a maximum range of 80 metres and a revolution frequency of 10Hz [90]. This setup generates around 130,000 points per frame. It also consists of two cameras that resemble the Point Grey Flea 2 models on either side of the LiDAR. These two cameras have a resolution of 1392 by 1024 pixels with a field-of-view of 72 degrees [90]. Because of the high degree of realism in the data acquired, this arrangement is a great stand-in for autonomous driving applications in the actual world.

The data's comprehensiveness includes seven separate environments, each of which represents a different driving condition. From Urban to Rural, and Highways to Mountains, these seven simulations cover each variety of locale. This setup therefore allows for a wide range of environmental conditions for testing of the localisation algorithms. Every environment consists of five thousand LiDAR frames. Each frame records precise spatial data such as the x, y and z coordinates along with the cosine of the angle at which the LiDAR beams are incident, and the time stamps that document the order in which the data was collected. The data also contains semantic annotations added to these frame data, which classify a variety of environmental elements, including automobiles and road markings (but we are not going to use the semantics).

In addition to providing raw point cloud data, the data was particularly selected to assist the construction of complex localisation algorithms. The "poses_lidar.ply" file contains the precise LiDAR locations. These pose files are essential because they serve as a ground truth for algorithm validation. These files contain the precise location and orientation of the LiDAR sensor during the data-gathering process. This study's methodology narrows the emphasis of the investigation to just LIDAR-based localization techniques by concentrating only on these LIDAR data points and pose files.

3.2 Algorithm Design

This section describes the approaches used to accurately estimate the location of the sensor on autonomous car. The methods used in this chapter are a combination of statistical, geometric, and machine-learning approaches. To create a strong framework for accurate and dependable sensor positioning, each subsection examines the various roles and procedures, highlighting how they relate to one another and the sequential flow that converts data into spatial insights.

3.2.1 Data Selection and Pre-processing

A. Loading Point Cloud Data

The algorithm starts with the loading of the point cloud data. This is essential for obtaining the spatial information required for localisation. These data points depicting the three-dimensional environment surrounding the vehicle, are obtained by the LiDAR sensors installed on the autonomous vehicles. The primary goal is to prepare and clean the raw sensor data. This helps increase the accuracy for subsequent processing. The `'load_point_cloud'` function is designed to parse the ply files extract the 'x' and 'y' coordinates, and convert them to pandas data frame for further processing.

B. Outlier Removal

After loading of the data, the next step is the removal of outliers. This is handled by the `'remove_outliers'` function. Outliers in the point cloud data can significantly skew the results of the spatial analysis which might lead to inaccurate estimations of the sensor's position. To resolve this, the function computes the Z-scores of the data points, which measures the deviation of each point from the mean in terms of standard deviations. Points with the Z-score above the set threshold are filtered out. This ensures that only those data points are retained which truly represent the environment thereby enhancing the robustness of the estimation process.

By ensuring that the data is clean and representative, the foundation is set for reliable and precise localisation, which is crucial for autonomous vehicles.

3.2.2 Data Transformation and Initial Analysis

A. Geometric Centre Calculation

Finding the point cloud's geometric centre, or centroid, is an essential step in locating the sensor in relation to its surroundings. To provide a basic approximation of the vehicles' position, the step computes the average coordinates (x, y) of all points in the point cloud. This is done by the `'calculate_geometric_center'` function. This centroid helps with the first approximation for iterative algorithms and serves as a guide for more intricate localization techniques.

B. Polar Coordinate Transformation

Cartesian coordinates (x, y) are converted to polar coordinates (r, θ) to reduce the computing complexity of subsequent algorithms. Polar coordinates are particularly useful in circular or rotating settings, which are common in autovnomous vehicle navigation around curves or barriers. These coordinates define a location in space with a radius and an angle. The conversion is carried out using the `'convert_to_polar_coordinates'` function. It calculates the angle θ as the angle concerning the positive x-axis and the radius r as the Euclidean distance from the geometric centre. To filter and segment the point cloud according to angular density or radial distance from the centre, this transformation is necessary.

This transformation is especially helpful when the algorithms call for analysing features according to their circular distribution or when angular deviation adjustments are required, e.g., localising elements in situations where direct line-of-sight measures are not possible. Like on a curving road or around the corners, we may require the identification of clusters or the computation of the dispersion of data points in particular angular segments.

The approach optimizes the effectiveness of the subsequent localisation operations. This is achieved by utilizing both the polar coordinate transformation and the geometric centre computation to prepare the data. The efficiency of clustering algorithms and other pattern recognition tasks—which are essential to autonomous vehicle navigation systems—is improved by these changes. It also makes it easier to comprehend the point cloud's spatial layout. Ensuring that the data used in this stage is properly prepared for geographic analysis, lays the foundation for the use of more advanced localization algorithms.

3.2.3 Position Estimation Techniques

Finding accurate sensor positions is essential for autonomous navigation. This approach optimizes sensor localization from LiDAR data by integrating many computational geometry and clustering algorithms. Methods such as minimal enclosing circles, convex hull, K-Means clustering, and iterative closest point adjustments are applied to the LiDAR data for initial estimation. Through the comparison and evaluation of typical localization procedures' accuracy and efficiency in different environmental contexts, these methods aid in identifying possible issues and areas that require further research.

A. Clustering and Position Estimation Using K-Means

This function groups a dataset into a certain number of clusters (given by `n_clusters`) using the K-Means clustering algorithm. K-Means clustering is very common technique used in machine learning. This approach is especially helpful in unsupervised learning when labels or categorizations of data points are unknown. The application of the K-Means clustering algorithm involves several distinct steps:

1. **Initialization:** K-Means randomly selects several points from the dataset as the preliminary centroids.
2. **Assignment:** Each point in the dataset is then assigned to the nearest cluster based on the Euclidean distance from the current centroids.
3. **Update:** After all points have been assigned to a cluster, the centroids for each cluster are recalculated. This is done by computing the mean of all points that have been assigned to each cluster; effectively updating the centroid to the centre of these points.
4. **Iteration:** The second and third steps are repeated iteratively. During each iteration, points are reassigned to clusters based on the newly calculated centroid. These centroids are then updated again. This process continues until the position of the centroids stabilizes and no further changes occur, or if the maximum number of iterations is reached.
5. **Result:** Once the final clusters are determined, the sensor position is estimated by calculating the mean of these centroids. The assumption here is that the actual sensor location should logically be at or near the central convergence point of the main features detected by the LiDAR, which is represented by these cluster centres.

This clustering method works effectively when the features in the LiDAR data are well separated or when the data points naturally form different groups. The resulting estimate offers a positional baseline for the sensor that can be further optimized or compared with alternative approaches to evaluate robustness and accuracy.

B. Convex Hull for Geometric Positioning

The smallest convex polygon or shape that can completely enclose a set of points in a place is known as a convex hull in geometry. The convex hull is especially useful when estimating the position of sensors since it may be used to determine the outermost bounds of data points that are gathered by sensors (like LiDAR or other scanning technologies). This method offers a reliable way to evaluate the overall

distribution and direction of the data by concentrating on the boundary created by these extreme points. There are several key steps in calculating the sensor's position using the convex hull method:

1. **Data Conversion:** The input consisting of 'x' and 'y' coordinates is converted into a NumPy array. This format is essential when performing complex geometric operations efficiently.
2. **Convex Hull Calculation:** With the help of the `'ConvexHull'` method from the `'scipy.spatial'` library, this function calculates the convex hull of the dataset.
3. **Extracting Hull Points:** Once the convex hull is calculated, the indices of the points forming the hull are retrieved. These points are termed as "hull points" and are the vertices of the convex polygon encapsulating all other points.
4. **Centroid Calculations:** The next step calculates the geometric centroid of these hull points. This is done by averaging the 'x' and 'y' coordinates of the hull points. The centroid is often used as a robust estimate of the central location of the entire dataset, particularly in uneven distributions.
5. **Output:** The function returns the coordinates of the centroid. This position is hypothesized to be an effective estimate of the sensor's location, as it is relatively insensitive to outliers, and it also does not depend heavily on the inner distribution of points within the hull.

The convex hull approach to sensor position estimation is especially useful in situations when data points form an irregular but dense cluster, as it offers a simple, non-parametric technique of centre calculation. Because of its resilience to outliers and different data densities, it's a useful tool for preliminary location estimates in a wide range of scenarios.

C. Minimum Circle Enclosing

The process of estimating the minimal enclosing circle for a sensor involves determining the smallest circle that can contain all of the data points that are provided. This method reduces the impact of outliers on positional calculations by guaranteeing that all points are inside a restricted area, which makes it especially helpvful for robust position estimation. The steps involved for minimum enclosing circle estimation are:

1. **Input Data:** This function accepts a data frame containing 'x' and 'y' coordinates of data points, typically derived from a sensor like LiDAR.
2. **Convex Hull Calculation:** First this method calculates the convex hull of the data using the `'ConvexHull'` function from the `'scipy.spatial'` module. The convex hull represents the smallest convex polygon enclosing all points. The vertices of this convex hull are extracted to form 'hull_points'.
3. **Centroid Calculation:** The next step involves the calculation of the geometric centroid of these hull points. This centroid represents the mean of the 'x' and 'y' values of the convex hull's vertices and serves as an approximation of the circle's centre.
4. **Radius Determination:** The radius of the circle is determined by calculating the maximum distance from the centroid to any vertices of the convex hull. This ensures the circle is just large enough to enclose all the data points.
5. **Output:** The function returns the coordinates of the centroid and the radius of the circle. These outputs can be used to visualize the circle or to further analyse the data's spatial properties related to the sensor's estimated position.

Because only the points on the hull affect the size and position of the circle, this method is resistant against outliers and yields a location estimate that is both conservative and comprehensive based solely on the outermost data points.

D. Iterative Closest Point (ICP) Adjustment

Refinement of sensor location estimates based on a series of observed data points is achieved by the sophisticated methodology known as the Iterative Closest Point adjustment method. This approach is especially helpful when there is a preliminary rough estimate of the position available and additional

refinement is required for precision. The steps involved in the calculation of sensor position using ICP adjustment are as follows:

1. **Input Data:** This function takes the same input as other functions, a data frame with 'x' and 'y' coordinates of the data points. However, along with the data frame, this function also requires an initial estimate of the position. In this case, this initial estimate is provided by the Robust Centre Estimation function. This function applies an Isolation Forest algorithm to detect and exclude outliers, ensuring that the initial position estimate is based on the most reliable data.
2. **Initial Position Adjustment:** The robust centre estimation function calculates the centroid of the remaining data points after removing outliers. This centroid serves as the initial estimate for the ICP adjustment. The data points are then translated such that this initial estimate is moved to the origin. It simplifies the subsequent calculations by aligning the data with the initial estimate.
3. **Nearest Neighbour Fitting:** The nearest neighbour model is fitted to the translated data points. This model searches for the closest data point in the original dataset to each point in the translated dataset. Thus, establishing correspondences based on spatial proximity.
4. **Point Matching and New Estimate Calculation:** For each point in the translated dataset, the nearest point from the original dataset is identified. The mean of these match points is calculated, providing a new estimate of the sensor's position. This mean averages out the location of the closest points, assuming that these represent the most likely the true position of the sensor relative to the initial estimate.
5. **Output:** This function in the end returns the new estimated position coordinates.

The robust centre estimation function plays a crucial role in the ICP adjustment function. It helps to filter out the anomalous data points that could skew the position estimation. This is done using an Isolation Forest which classifies each point as an inlier or an outlier based on the surrounding density of points. The remaining inlier points are used to calculate a preliminary centroid. This helps provide a robust initial guess that is less affected by extreme values or dispersed data clusters.

When sensor data is noisy or contains outliers, the technique of employing a robust method for initial estimation and then adjusting the ICP is especially useful. It guarantees that the position estimation is improved by iterative refinements and is built on a strong statistical foundation, improving the accuracy and dependability of sensor positioning in challenging situations.

3.2.4 Advanced Mathematical Modelling

Traditional techniques such as K-Means Clustering, Convex Hull, Minimum Enclosing Circle and Iterative Closest Point (ICP) provide a basic framework for predicting sensor position. However, these approaches may not work in real-life scenarios. In conditions where anomalies and dynamic environmental elements predominate, or where the data may not surround the sensor position equally, these approaches fail. When it comes to accurate localization in a variety of terrains and among obstacles, such conventional approaches usually fall short of capturing the subtle but crucial characteristics of spatial distribution.

To close this gap, advanced mathematical modelling uses methods that go **deep**er into the structural subtleties of the data. Basic geometric methods can miss important environmental changes like borders or boundaries. These can however be highlighted by the calculation of first and second derivatives in the data. In addition, the model can be made to adjust dynamically to local fluctuations. This can be achieved by the techniques such as the Savitzky-Golay filter which dynamically adjusts the parameters for smoothening of the data. By doing this, noise is reduced and significant features are highlighted, resulting in a more reliable and accurate positional estimate for the sensor.

These improved methods focus on important shifts in the data landscape and adjust to its dynamic nature. Because of this, the accuracy and reliability of sensor location predictions are greatly increased. Advanced mathematical models provide a thorough grasp of the sensor's environment by identifying exact changes in spatial configurations and smoothing data based on its local properties. By doing this,

the shortcomings of conventional approaches are addressed. It also strengthens the system's resistance to abnormalities in the environment and inconsistent data.

A. Dynamic Savitzky-Golay Parameter Adjustment

A digital filter called the Savitzky-Golay filter smooths collected digital data points using a polynomial algorithm. This filter improves the accuracy with the help of the least square approach which fits the consecutive subsets of nearby data points with a low-degree polynomial. This method preserves important statistical characteristics like relative maxima, minima and width. This is achieved by balancing the output signal's smoothness against the original data's fidelity.

The steps taken to implement this filter in our approach are as follows:

1. **Initialization:** To start the optimization process, the function first defines an extremely high initial score (float('inf')), ensuring that any practical score that is generated is lower. It also sets up placeholder variables for storing the best window length and polynomial order.
2. **Parameter Iteration:** The function iterates through a series of potential window lengths. This ensures that each window is odd-numbered because the filter requires an odd-sized window to symmetrically distribute the fitted polynomial. This loop starts from 11 until the user-defined maximum window size, increasing by 2 each time to maintain the odd numbers. The function also iterates through a small set of polynomial orders (typically 2, 3, and 4) for each window length. These orders represent the complexity of the polynomial fit. Higher orders can fit more complex data structures but might lead to overfitting in smoother data.
3. **Filter Application and Error Calculation:** Within each nested loop, the Savitzky-Golay filter is applied to the data using the current parameters. The output is a smoothed version of the original dataset. The mean squared error (MSE) between this smoothed dataset and the original dataset is computed. This error represents how well the polynomial has managed to capture the underlying trends without fitting the noise or other fine-scale structures that are not of interest.
4. **Optimization:** If the newly calculated MSE is lower than the previously recorded best score, the function updates its record of the best score and the associated parameters. This step ensures that by the end of the iteration process, the parameters that lead to the least distortion of the underlying signal while reducing noise are selected.
5. **Output:** The function returns the optimal parameters found during the iterations. These parameters are ideal for smoothing the data under current conditions. These are then used in subsequent data processing steps to ensure that the data smoothing is as effective as possible without losing critical information.

This approach tailors the smoothing process to the specific noise characteristics of the sensor data and also enhances the robustness and accuracy of the localisation estimates derived from this data, by dynamically adjusting the parameters of this filter.

B. Derivative Analysis

This function utilizes the derivative analysis to identify subtle yet significant variations within the data, which could signify environmental boundaries or unique features. This analytical step is crucial for refining position estimates in complex environments. The steps involvved in this function are:

1. **Data Preparation:** First the input data is sorted based on the angular component 'theta'. This arranges the data continuously, thus facilitating meaningful derivative calculations. Sorting data according to theta guarantees that subsequent computational operations, like calculating derivatives, adhere to the data's natural order as it could exist within the circular field of view of the sensor.
2. **Dynamic Filtering:** Before the calculation of derivatives, the function optimises the smoothing process through the `'dynamic_savgol_paramters'` function which selects the best window length and polynomial order for the Savitzky-Golay filter. Using these dynamically determined parameters, the function smooths the radial and angular data by applying the filter. Smoothing

is important because it lowers noise and makes it possible to identify patterns and changes more clearly without being misled by noise or anomalies in the data.

3. **Derivative Computation:** The first derivatives of the smoothed radial and angular data are calculated using the np.gradient function. These first derivatives represent the rate of change in distance and angle. This helps us analyse how quickly the sensor's environment is changing in both dimensions. Similarly, the second derivatives are calculated for both the components. These represent the acceleration of changes. This helps us pinpoint where changes are beginning to occur or where they are most pronounced.
4. **Angular Wrap-around Handling:** Angular data is cyclical. It means that it represents directional information that wraps around at 360 degrees (or 2π radians). This function includes a specific adjustment step where is checks for discontinuities in the angular derivatives that occur due to wrapping from 2π back to 0. It adjusts these values to maintain the continuity. This ensures that the derivative calculations do not mistakenly interpret a natural wrap-around as a significant change.

Through precise computation and examination of these derivatives, the function offers an in-depth quantitative evaluation of how the spatial properties of the surroundings are evolving around the sensor. This information is very helpful in locating important elements that are necessary for accurate localization, such as corners, edges, or other environmental limits.

C. Integration into Sensor Position Estimation

The `'estimate_sensor_position'` function combines several complex mathematical techniques to provide a precise estimate of the sensor's position. This function involves the following steps:

1. **Initial Estimate:** The process begins with an initial guess that is based on the geometric centre of the dataset. This centre is calculated as the mean of the x and y coordinates of the data points. This step establishes a baseline from which more nuanced analyses can refine this estimate.
2. **Refinement through Angular Analysis:** This initial estimate is then enhanced by analysing the angular distribution of the data. It identifies peak in this distribution which represents the angles where the data points are most densely clustered. These peaks often correspond to significant environmental features such as walls or edges that occur within the sensor's range. By focusing on these dense areas, the function assumes that these angles are more likely to be aligned with the real environmental features, rather than arbitrary distribution of points. This step refines the estimate by weighting the sensor's position towards these features, enhancing the accuracy of the localization.
3. **Utilization of Derivatives:** The next level of refinement uses the first and second derivatives of the radial and angular data which were calculated earlier. Areas with high derivatives indicate significant changes in the environment such as boundaries, corners, or changes in landscape. By focusing on these areas, the function adjusts the estimated position to align more closely with the detected features. This method leverages the detailed textural information contained within the data's derivative structure to further pinpoint the sensor's position.
4. **Error Calculation:** In the end, the function quantifies the accuracy of its estimations. This is done by calculating the Euclidean distance between the refined estimated position and the actual ground truth position. This distance serves as a direct metric of the estimation accuracy and provides a clear measure of how closely the estimated position matches the true location of the sensor.

3.2.5 Performance Evaluation and Improvement Analysis

The last stage of the localization process is about measuring the gains over earlier estimates and assessing how well the techniques that were used performed. This phase is essential for incremental improvements and verifying the algorithms' efficacy. This process is structured as follows:

A. Improvement Calculation

This function serves as the cornerstone for assessing enhancements in localisation accuracy. It compares new position estimates against previous results and a known ground truth.

1. **Data Loading:** The function first loads the ground truth data along with the previously estimated positions from earlier runs.
2. **Error Calculation:** For each method, the function calculates the Euclidean distance between the estimated positions and the ground truth, deriving an error metric for both old and new estimates.
3. **Improvement Metrics:** Improvements are quantified by comparing the old errors with the new errors, and calculating the percentage improvement. This provides a clear metric of how much the new methods have enhanced the accuracy of the sensor's localization.

B. Improvement Aggregation and Display

The `'consolidate_improvements'` function aggregates all the improvement data across different runs or scenarios. This function is useful when dealing with multiple datasets/towns in a simulated environment.

1. **Data Consolidation:** This function compiles improvement data from various sources into a comprehensive data frame. It structures the data by method and scenario, facilitating an organized overview of performance across different conditions.
2. **Enhanced Readability:** By consolidating data, it allows for easier comparison and analysis, providing a holistic view of the system's performance.

The `'display_improvements_table'` function is designed to visually present the consolidated improvements data. It utilizes data pivoting and table formatting to create an easily interpretable display of the improvements.

1. **Visualization:** This function typically outputs a table that categorizes improvements by method and location or scenario, highlighting the effectiveness of each approach in different environmental conditions.
2. **Utility:** It helps in identifying the methods that are performing better than others and where further adjustments or optimizations might be needed.

Chapter 4

Implementation

Understanding how theoretical ideas and methods are used in practice to provide accurate localisation results is crucial and this provides a thorough explanation of how. It describes the software environments and tools that were utilized. It also touches on the methodical order in which the functions are executed and the reasoning behind the order in which they were performed. This part guarantees process transparency by outlining the flow from data ingestion and preprocessing to sophisticated mathematical modelling and performance evaluation. It also demonstrates the methodical methodology used to maximize localization accuracy in various settings. This thorough explanation helps evaluate the robustness and scalability of the localization system in addition to helping to replicate the study.

4.1 Software and Tools Used

The project leverages Python. Python is a versatile programming language widely acclaimed for its comprehensive libraries supporting data analysis, machine learning, and scientific computing [94]. This technological stack supports the intensive computational demands of the project. It also enhances the workflow through its integrated capabilities for data handling, processing, visualisation, and machine learning. The key libraries used in this project are described below.

4.1.1 NumPy and Pandas

NumPy is fundamental for high-performance scientific computing. It provides support for large, multi-dimensional arrays and matrices, along with a vast collection of mathematical functions to operate on these arrays. With localisation jobs, managing massive datasets and frequent operations on enormous quantities of coordinates and sensor readings, this capability is essential.

Pandas provides operations and data structures for working with time series and numerical tables. This library is useful for prepping the intricate datasets in this research since it makes activities like data filtering, transformations, and aggregation simpler. To prepare the inputs for localization algorithms, data must be merged, reshaped, and summarized. This is where its data frame form comes in handy.

4.1.2 Matplotlib and Seaborn

Matplotlib is a comprehensive library. It is used for creating static, animated, and interactive visualisations in Python. It is used extensively throughout the project to plot graphs, which help in analysing the performance of the localisation methods and understanding underlying patterns in the data.

Seaborn is built on top of matplotlib. It integrates closely with pandas data structures and provides a high-level interface for drawing attractive and informative statistical graphics. It enhances the visualisation capabilities by allowing for a more convenient graphical representation. This is particularly helpful for comparing the effectiveness of different localisation techniques.

4.1.3 Scikit-learn

This library is used for implementing machine learning algorithms efficiently. Being built based on the libraries like NumPy, SciPy, and matplotlib, it offers straightforward and effective tools for data mining and analysis. Scikit-learn plays a key role in this project for clustering outlier detection, especially for techniques like K-Means and Isolation Forest.

4.1.4 Plyfile

The Plyfile library is a Python module for reading and writing the PLY files. These files are commonly used to store three-dimensional data from laser scanners and other 3D imaging systems. Plyfile is essential for this project since it allows for easy import and export of PLY files containing LiDAR point cloud data.

4.2 Code Structure

The project follows a procedural flow of code execution for each town. This section delves into the comprehensive breakdown of the function execution and rationalization for each town.

4.2.1 Initial Setup and Data Acquisition

The directory for each is set as follows - `town_directory = {path_to_each_town_directory}`, and the path to the ground truth is defined. This ground truth contains precise locations which serve as a benchmark to evaluate the accuracy of estimated positions. After that, `'plot_random_lidar_with_pose(town_directory)'` is called to visualize LiDAR data points alongside the actual sensor position (ground truth). This function selects a random LiDAR file, loads its data, and plots it against the ground truth positions. This provides a visual insight into the data distribution and initial sensor accuracy.

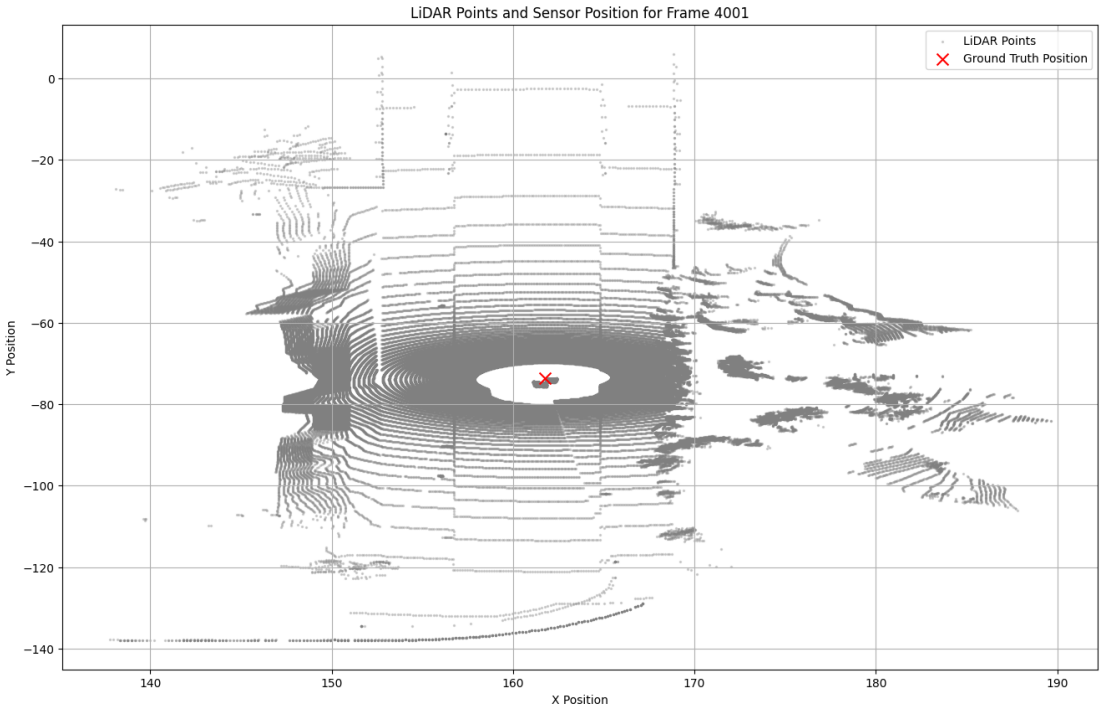

Figure 4: *Town 01: LiDAR Cloud Points and the Actual Sensor Position ('X' Mark)*

4.2.2 *Initial Error Analysis*

The function `'plot_estimations_and_compute_errors(random_filepath, ground_truth)'` computes the localisation using basic methods like K-Means, Convex Hull, etc. and plots these alongside the ground truth with the LiDAR data points.

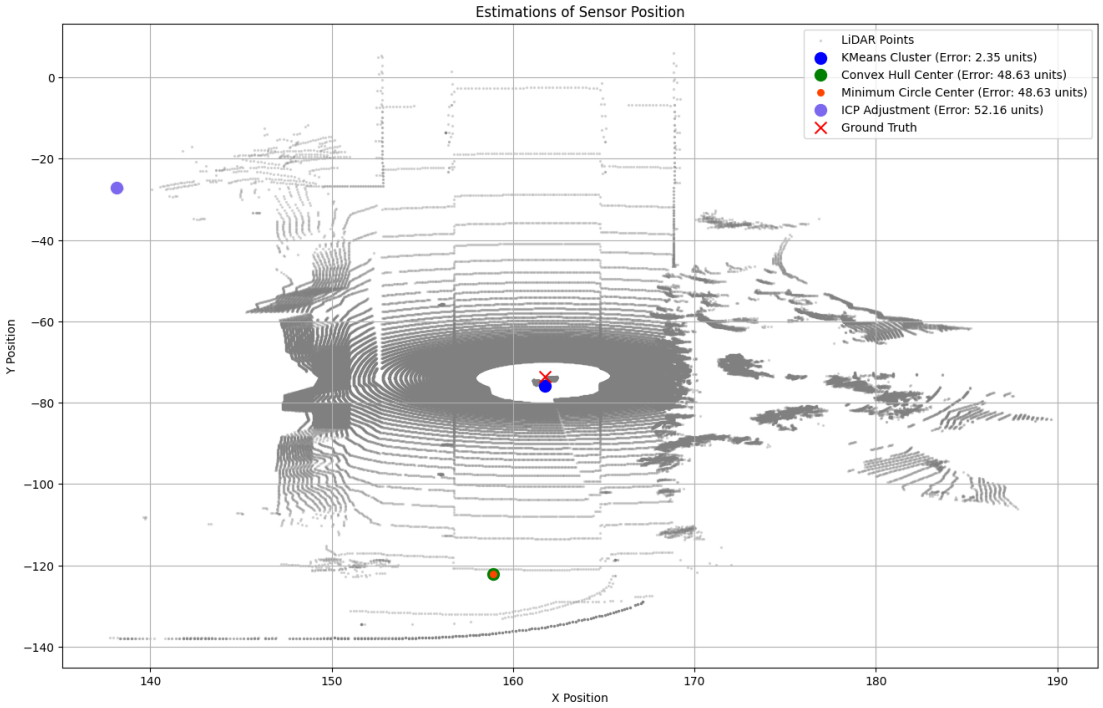

Figure 5: *Town 01: Localisation using basic methods*

This step evaluates the initial performance of basic methods first before moving on to the mathematical approach. The function also calculates the errors for each method and passes it to the function `'plot_error_graph(errors)'`. This function plot the graphs as seen below.

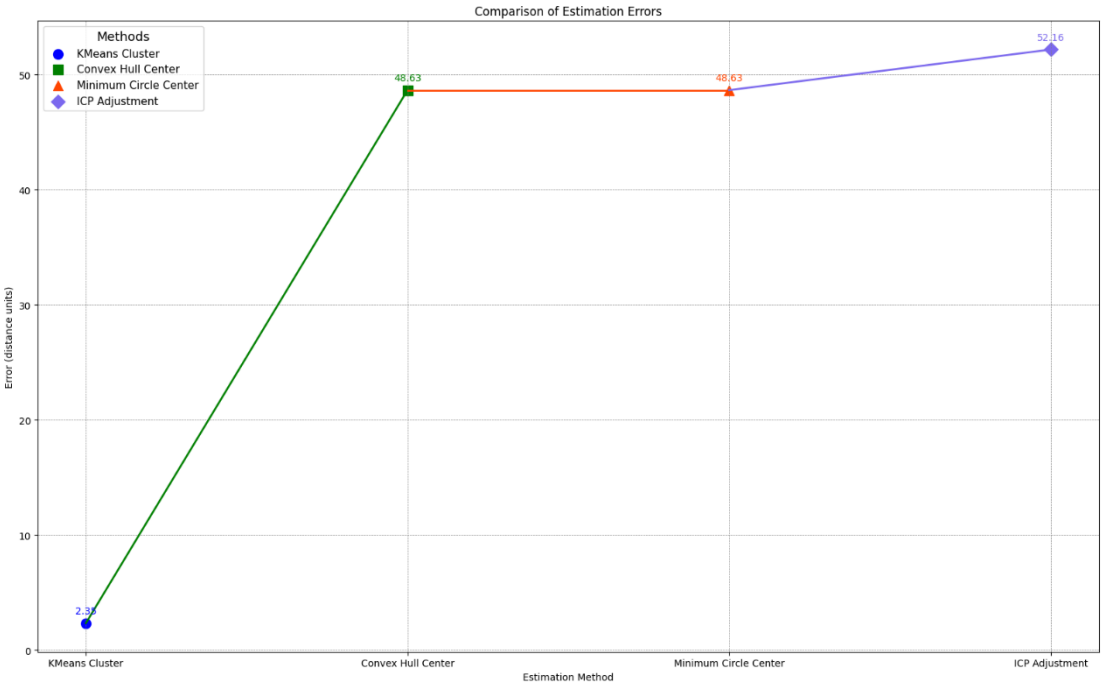

Figure 6: *Town 01: Comparison of the Estimation Errors for different methods*

4.2.3 Path Visualisation

Using the `'plot_ground_truth'` function, the code visually represents the path followed by the car or sensor according to the ground truth data. This visualization is crucial for understanding the movement dynamics and the environmental context within which the sensor operates.

4.2.4 Batch Processing

The `'batch_process_estimation'` function processes multiple LiDAR files for the respective town to estimate the positions using the basic techniques mentioned in the methodology chapter. It aggregates these estimations to analyse trends. The estimations for each technique are plotted with the ground truth path as an overlay. This helps us visualise how accurately each method estimates the location of the sensor. The `'plot_error_statistics'` function plots the errors captured in the batch estimation function, as a line graph with the 'x' axis being the frame index and the 'y' axis being the error distance (calculated using Euclidean distance). This provides a detailed look at the performance across multiple data points.

4.2.5 Advanced Mathematical Modelling for Sensor Location

This part of execution uses the advanced mathematical modelling to accurately predict the location of the sensor. First, the `'load_point_cloud'` function loads the data from the ply file and the function `'remove_outliers'` removes the outliers from the data. After this, the function `'calculate_geometric_center'` is used to determine the geometric centre. This gives us an initial approximation of the sensor's location and helps in setting the baseline for further improvement.

The data is then transformed to polar coordinates to fit the circular or rotating patterns. Such patterns are generally found in many LiDAR datasets. The derivatives are computed using the `'compute_derivatives'` function which help us understand any important borders or characteristics. Using the `'estimate_sensor_position'` function, the location of the sensor is triangulated. This improves the localisation accuracy based on the dynamic changes in the data.

4.3.6 Performance Evaluation

After the calculation of the estimated position using the mathematical model, the final estimated position is plotted for the previously selected random ply file using the `'plot_estimated_position'` function. The provides visual and quantitative evidence of the improvement in localisation accuracy due to the advanced mathematical modelling. The function `'process_and_plot_estimated_path'` plots the estimated path using the mathematical model alongside the ground truth path. This helps us visualise the accuracy of the model over the whole dataset.

Using the `'plot_error_statistics_mathematical_approach'` function, similar to `'plot_error_statistics'`, the error line graph is visualised which provides insight into the error distance between the estimated sensor position and ground truth position. Next, the function `'calculate_improvements'` computes the improvements over the initial estimates from the basic functions and is consolidated using the `'consolidate_improvements'` function. This stage measures the improvements in localization precision and illustrates how well the techniques used worked.

Every step is carefully carried out for every town. This guarantees that the sequence maximizes the localization process while simultaneously preserving consistency across various datasets.

Chapter 5

Results and Discussion

The analysis across all the towns (Town 01 to Town 07) highlights significant improvements and consistent trends in localisation accuracy. The use of advanced mathematical model over the traditional localisation methods like Convex Hull, K-Means, ICP and Minimum Enclosing Circle demonstrate a substantial reduction in average error.

Key Findings:

- **Consistency Across Towns:** Similar patterns can be seen in every town with a mathematical approach greatly outperforming the conventional techniques. The mathematical approach consistently produced the lowest average errors. This demonstrated its robustness and effectiveness in a range of environments.
- **High Improvement Rates:** The improvement percentages using the mathematical approach across all the towns are exceptionally high. For example, in Town 01, the mathematical approach outperformed the Convex Hull and Minimum Circle method by reducing the errors by around 87%, and for the ICP Path, the errors were reduced by around 95%. This pattern can be seen to be consistent across other towns as well. The developed approach shows improvements ranging from 72% to 96%.
- **Reduction in Average Errors:** The developed mathematical approach reduced the average errors to as low as 2.46 in Town 07 and 3.43 in Town 05. This indicates a significant enhancement in precision. This significant decrease in error highlights how well the approach can interpret the data and use it for accurate localization.

Consistent Trends:

- **Variability among Traditional Methods:** Methods like the K-Means Path showed varying levels of accuracy. The average errors for the K-Means path range from about 7.51 in Town 03 to 11.59 in Town 01. However, these methods consistently showed less accuracy when compared to the advanced mathematical approach.
- **Superior Performance of Mathematical Approach:** The mathematical approach consistently managed to achieve the lowest average errors outperforming the basic localisation methods. This makes it a preferable choice for high-accuracy localisation needs in autonomous vehicle navigation and other related applications.

5.1 Ground Truth Paths

The ground truth graphs offer a visual depiction of the real routes that the sensor took through the places. These graphs are used as a standard to compare the accuracy of different localisation methods.
The ground truth graphs provide an accurate and transparent benchmark against which the estimated pathways are evaluated, by depicting the sensor's journey in two-dimensional space. Understanding the physical movement of the sensor in various urban layouts and conditions is made possible by this depiction. This also makes it easier to compare the actual movement of the sensor with estimated trajectories and assess how effective the localisation techniques were.

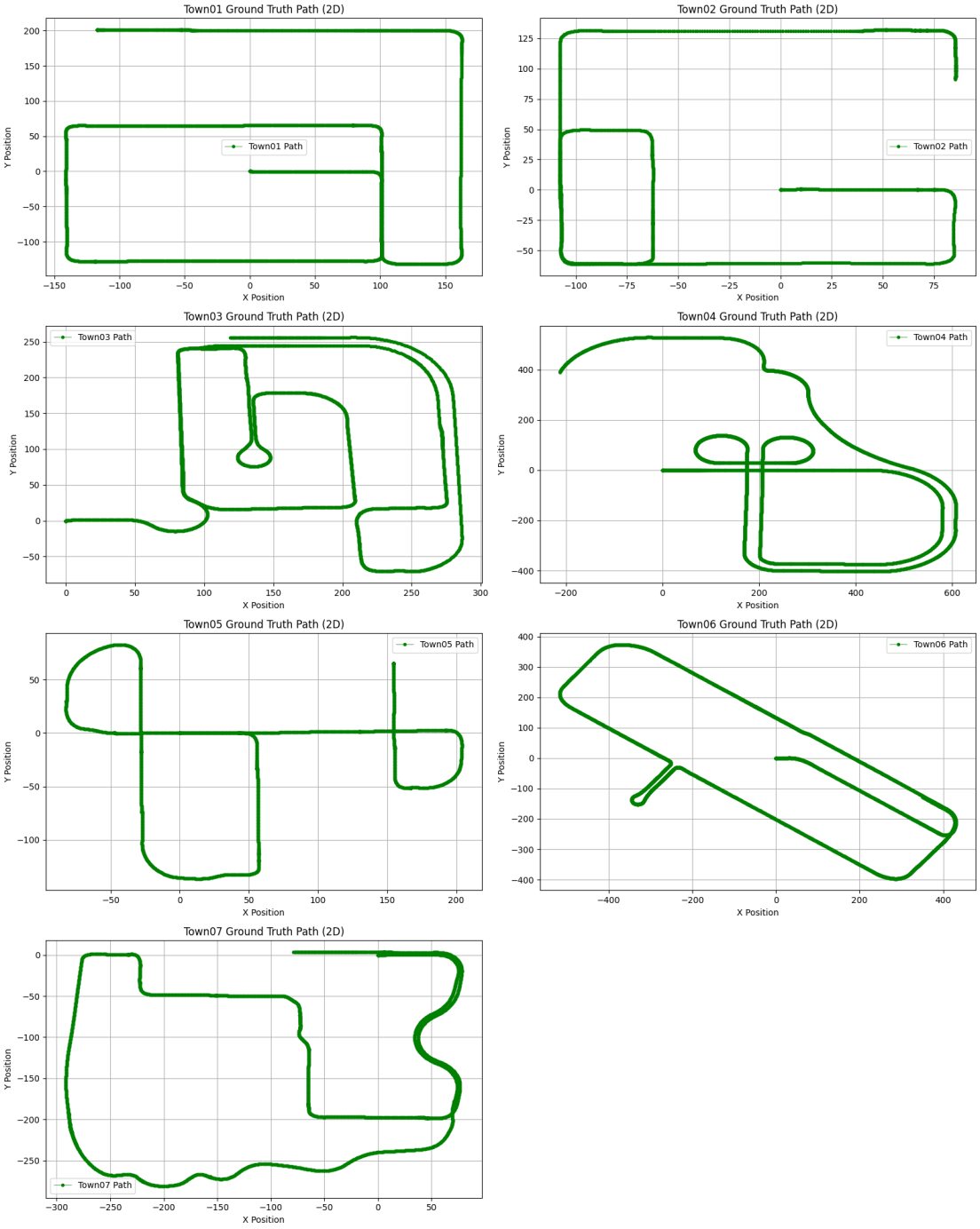

Figure 7: *Ground Truths for Towns 01 to Towns 07*

5.2 Estimated Paths and Error Graphs for Different Localisation Methods

In the assessment of the estimates path using different localisation methods, each method was compared to the ground truth path to determine the accuracy. The estimated paths highlight the accuracy of each localisation method under varying environments. The estimated path plots and error graphs are added in appendix A.

5.2.1 Estimated Paths

A. **K-Means Path:** When the estimated paths for different towns are compared with their respective ground truths, it is observed that the K-Means path exhibits a generally coherent tracking of the overall trajectories. However, there is a noticeable deviation and noise, especially in the areas on complex movements or where the paths loop back on themselves. These visualisations illustrate the algorithm's capacity to follow the main path but its limitations in precisely locating the sensor. The paths are riddled with jitters and extraneous branches indicating misclassifications by the clustering process. These results highlight the inherent challenges in using K-Means for precise localization tasks in dynamic environments, showcasing its sensitivity to the density and distribution of the input data.
B. **Convex Hull Path:** The convex hull method of estimating the sensor position can be seen to have a lot of errors. It deviates substantially from the ground truth. The errors can be majorly seen in the areas where the actual path involves tight turns of overlaps. Because of the nature of the convex hull method to encompass all the points, it may oversimplify the actual sensor movements and fail to capture the fine navigational details effectively.
C. **Minimum Circle Path:** This method shows a similar estimation to the convex hull method. It lacks precision in tracking the exact route. This is visible in the areas with back-and-forth movements or tight loops.
D. **ICP Path:** The ICP method appears to have the most erratic performance. While some sections, although a very small part, align with the ground truth, most of the estimated positions are dramatically off course. This inconsistent accuracy may be because of the method's sensitivity to initial alignment or potential susceptibility to specific types of noise.

5.2.2 Error graphs

A. **K-Means Path:** The error graph for this method demonstrates a high level of volatility with peaks frequently exceeding the 10-unit error mark. While the K-Means method may occasionally align with the ground truth, it often diverges significantly. This suggests that even though K-Means can capture general movement trends, it struggles with precision and consistency, particularly in complex environments.
B. **Convex Hull Path:** The graph for this method shows a wide variability. The error values can be seen clustering between 30 to 40 units. This indicates that the convex hull path method struggles extensively even in normal scenarios. There can be seen a high number of error peaks.
C. **Minimum Circle Path:** Similar to the errors seen in the convex hull path, this method's errors are also varied. It is seen to mirror the performance of the convex hull method. Similar to the previous method there can be seen a wide number of error peaks. This suggests that the Minimum Circle approach can be useful for broad localisation tasks. But it oversimplifies the trajectory and misses the navigational details.
D. **ICP Path:** The most dramatic and varied errors can be seen in the ICP method. These errors show significant discrepancies. We can see that the error values are mostly clustered around 75 to 80 units. It is way off the charts. This suggests that the ICP method is very volatile and cannot be used for localisation.

5.3 Advanced Mathematical Model Estimations

The advanced mathematical model integrates sophisticated mathematical techniques such as derivative analysis and dynamic parameter adjustment. This allows for effective adaptation to complex data characteristics and enhances precision. As seen from the results below, the path plotted from the estimated position by the mathematical model aligns remarkably with the ground truth, highlighting the model's robustness and accuracy in estimating sensor paths. This advanced model closely follows the ground truth along different sections of the journey, in contrast to simpler approaches that frequently deviate from the true path, particularly in areas involving complex manoeuvres. As seen from the results, this model performs consistently across varied environments. This proves that the method is extremely reliable for sensor path estimation in diverse settings.

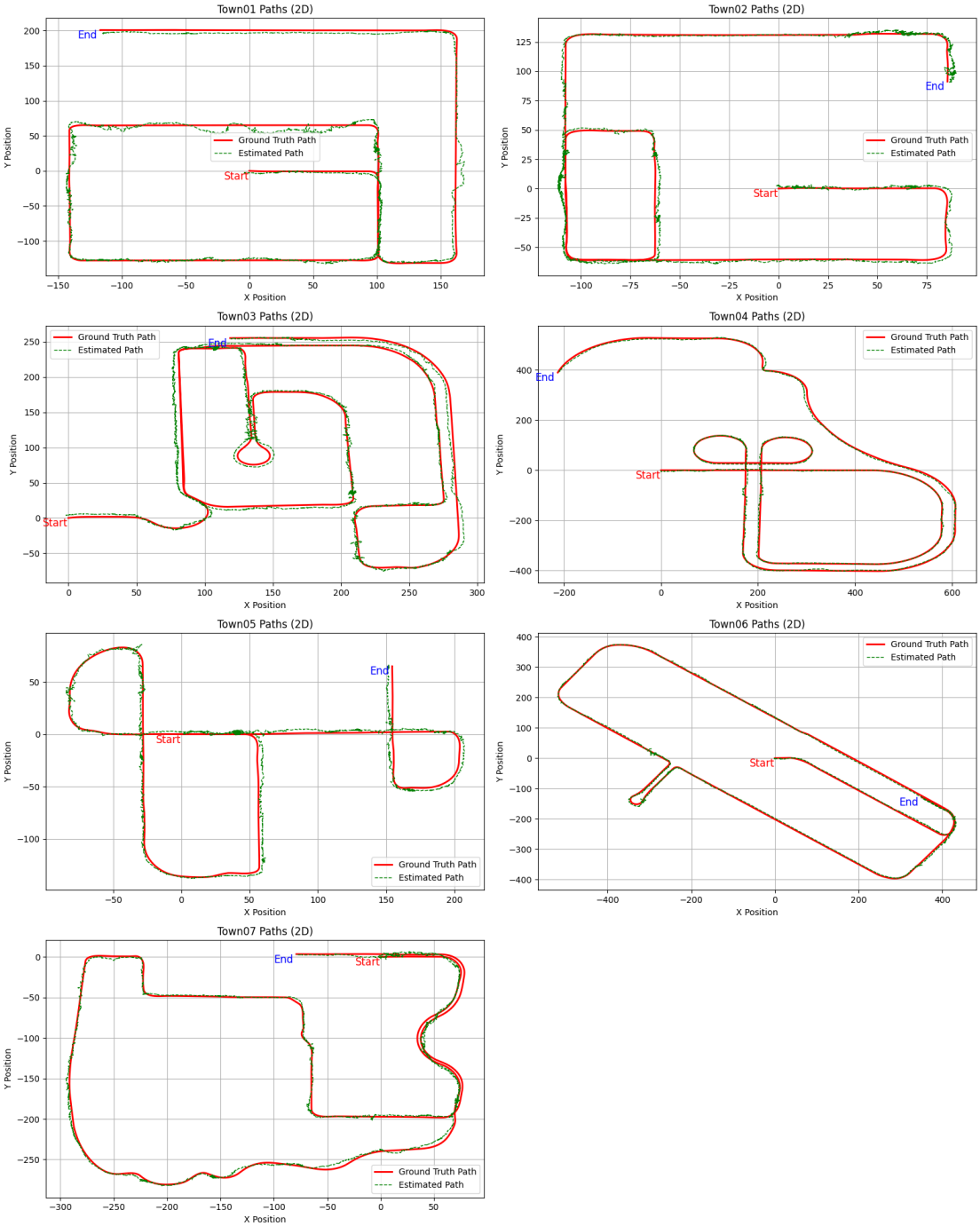

Figure 8: *Path as plotted by the estimations from the advanced mathematical model*

5.4 Improvement Analysis across all Towns

The table depicts the enhanced accuracy of the advanced mathematical model over the traditional methods across seven towns. The percentage improvement shows significant gain, especially over the ICP Path method, where the improvements are consistently over 95% in several towns. This indicates the model's ability to mitigate errors that these basic methods struggle with.

Town	Method	Average Error	Mathematical Model Improvement (%)
Town 01	Convex Hull Path	27.700720	86.965448
	ICP Path	73.343266	95.077033
	K-Means Path	11.598114	68.868517
	Minimum Circle Path	27.700720	86.965448
Town 02	Convex Hull Path	23.304435	87.981047
	ICP Path	65.077307	95.695967
	K-Means Path	09.410052	70.234500
	Minimum Circle Path	23.304435	87.981047
Town 03	Convex Hull Path	23.542194	86.750882
	ICP Path	73.993985	95.784613
	K-Means Path	07.510255	58.468347
	Minimum Circle Path	23.542194	86.750882
Town 04	Convex Hull Path	32.182132	90.432459
	ICP Path	73.872324	95.831946
	K-Means Path	11.144768	72.372340
	Minimum Circle Path	32.182132	90.432459
Town 05	Convex Hull Path	31.074955	88.956469
	ICP Path	58.266062	94.110170
	K-Means Path	07.981624	57.004083
	Minimum Circle Path	31.074955	88.956469
Town 06	Convex Hull Path	26.561291	88.959297
	ICP Path	75.321287	96.106608
	K-Means Path	09.547828	69.285652
	Minimum Circle Path	26.561291	88.959297
Town 07	Convex Hull Path	23.157344	89.351047
	ICP Path	67.316382	96.336680
	K-Means Path	08.815697	72.027002
	Minimum Circle Path	23.157344	89.351047

Figure 9: *Improvement Analysis across all Towns*

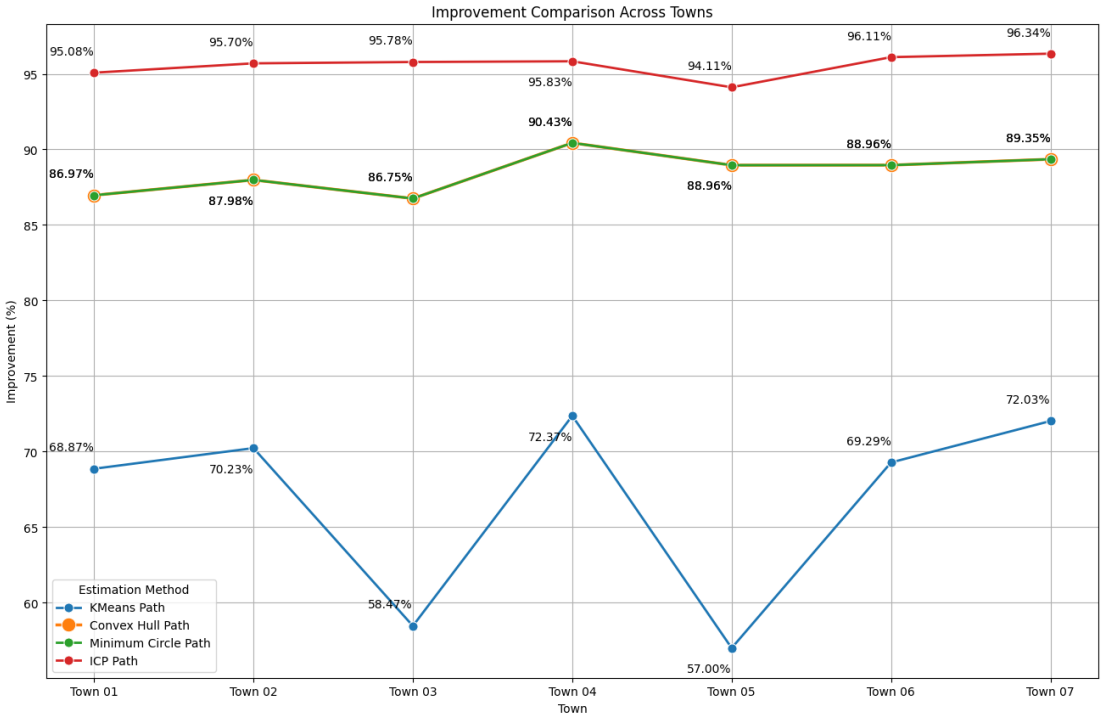

Figure 10: *Improvement Comparison across Towns*

5.4.1 Convex Hull Path

In the beginning, the Convex Hull methods reported average error rates of between 23.16 and 32.18 units for different towns. This technique is not very accurate with the internal complexities of the lines of the sensor, which could have varied even more from the real sensor line. The variations in the mathematical model involved more of a spatial analysis approach and they worked to reduce these errors considerably with percentages obtaining improvement rates from about 86.97% to 90.43%. This shows that even though there is a rough approximation of the problem accompanying the convex hull, there exists a mathematical model that helps to accommodate for the changes within the data that were previously ignored.

5.4.2 ICP Path

The method had a more average error than most adjustments as it underperformed by 95% as compared to the mathematical model in all towns. This demonstrates the sensitivity of the ICP method to initialization as well as the iterative characteristics of the ICP method, which may lack proper convergence for certain poses of LiDAR data. The approach employing the mathematical formulation incorporates the improvement components relevant to the problem of initial alignment. Thus, yielding a more accurate and dependable estimate which resulted in lower error rates.

5.4.3 K-Means Path

The K-Means method had the least initial errors when compared to the traditional methods. The errors ranged from 7.51 to 11.60 units which indicates that this method was reasonably good in classifying LiDAR points to outline the path motion of the sensor. However, the mathematical model still showed great improvements with percentages ranging from 54.87% to 72.37%. This signals that in K-Means, the clusters are the most robust and thus, K-Means can only capture the important clusters or ‘centres’ in the data. However, the mathematical model makes better use of these clusters using other contexts or sequ ences which K-Means may not consider.

5.4.4 Minimum Circle Path

This technique which determines the smallest circle that encloses all points matched the results and advances of the Convex Hull method. The method primarily reduces the path determination to simply a geometric problem. This does not accomplish the details of the intricate path, particularly in the case of paths with curves or turn angles. The mathematical model showed improvements similar to the Convex Hull method with percentages around 86.95% to 90.43% improvement.

Chapter 6

Conclusion and Future Work

The results from the advanced mathematical model have shown better accuracy of localisation than classic techniques such as Convex Hull, K-Means, Minimum Enclosing Circle, and Iterative Closest Point (ICP) methods. In all the towns, this model not only diminished average errors but also showed peak rates of improvement, being 72% to over 96% more effective than standard techniques. This means that in practice, there is a massive improvement in accuracy. It proves that the model can properly analyse complex LiDAR information and its efficiency in different environmental conditions.

Contributions to the Field

This research addresses the issue of autonomous vehicle navigation by proposing a constructive and reliable mathematical model that improves sensor localisation significantly in situations where a GPS signal is unavailable. The method developed and tested in the frame of this dissertation provides a solution that can be applied not only in automotive systems but also in many real-life systems such as robotics and mobile mapping systems.

Limitations of the Study

Regardless of the above successes, the study has its shortcomings. The major limitation is that it is based on simulated data which, although rich and detailed, is limited in recreating actual driving conditions to some extent. Moreover, the complexity of the advanced models may also restrict their use in actual practice without further enhancement or improvement in processing power and hardware.

Future Research

Several key points can be improved upon in the future.

1. **Real-World Testing:** Applying these models for real-world testing with the intent to obtain results that are different than those predicted and to verify the models, based on the actual, non-simulated data.
2. **Algorithm Optimization:** Possible strategies can be employed to make the model more optimized in terms of speed and precision. This can be used in the real-time environment in autonomous vehicles.
3. **Sensor Fusion:** This method can be integrated with other sensors such as radar and/or video so that localisation can be improved in terms of accuracy and also in terms of robustness.
4. **Machine Learning Integration:** Machine Learning methods can be used in many ways to imp rove the model even further with the possibility of parametrizing this model dynamically when new environmental information is available. Machine learning can also be used to tune the model even further by comparing it to the location provided by the GPS when the GPS strength is highly reliable.

In conclusion, the model introduced in this project marks a great advancement in the area of sensor localisation technology. However, there is a region that warrants further advancement for further penetrating and optimizing the rapidly changing world forms of technology. This opens up new perspectives for more concentrated and thorough research that one day will be at the base of the commercial application of autonomous systems.

References

[1] R. Hussain and S. Zeadally, "Autonomous cars: Research results, issues, and future challenges," *IEEE Communications Surveys and Tutorials*, vol. 21, no. 2, pp. 1275-1313, 2019, doi: 10.1109/COMST.2018.2869360.

[2] P. J. Besl and N. D. McKay, "A method for registration of 3-D shapes," in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 14, no. 2, pp. 239-256, Feb. 1992, doi: 10.1109/34.121791.

[3] A. V. Segal, D. Haehnel, and S. Thrun, "Generalized-ICP," in *Robotics: Science and Systems*, MIT Press Journals, pp. 161-168, 2010, doi: 10.15607/rss.2009.v.021.

[4] P. Biber and W. Strasser, "The normal distributions transform: a new approach to laser scan matching," in *Proc. 2003 IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS 2003)*, Las Vegas, NV, USA, 2003, pp. 2743-2748 vol.3, doi: 10.1109/IROS.2003.1249285.

[5] K. Yoneda et al., "Lidar scan feature for localization with highly precise 3-D map," in *2014 IEEE Intelligent Vehicles Symposium Proceedings*, Dearborn, MI, USA, 2014, pp. 1345-1350, doi: 10.1109/IVS.2014.6856596.

[6] B. Charmette, E. Royer, and F. Chausse, "Vision-based robot localization based on the efficient matching of planar features," *Machine Vision and Applications*, vol. 27, pp. 415-436, 2016, doi: 10.1007/s00138-016-0759-5.

[7] SAE International, "Surface Vehicle Recommended Practice Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles," 2021. Available at: https://www.sae.org/standards/content/j3016_202104/ [Accessed Aug. 24, 2024].

[8] J. Kocić, N. Jovičić, and V. Drndarević, "Sensor and Sensor Fusion in Autonomous Vehicles," 2018. Available at: https://doi.org/10.1109/TELFOR.2018.8612054.

[9] A. Davies, "Turns Out the Hardware in Self-Driving Cars Is Pretty Cheap," *WIRED*, 2015. Available at: https://www.wired.com/2015/04/cost-of-sensors-autonomous-cars/ [Accessed Aug. 24, 2024].

[10] D. Nister, O. Naroditsky, and J. Bergen, "Visual odometry," in *Proc. 2004 IEEE Computer Society Conf. on Computer Vision and Pattern Recognition (CVPR 2004)*, Washington, DC, USA, 2004, pp. I-I.

[11] Z. Sun, G. Bebis, and R. Miller, "On-road vehicle detection: a review," in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 28, no. 5, pp. 694-711, May 2006.

[12] L. Fridman, "Human-Centered Autonomous Vehicle Systems: Principles of Effective Shared Autonomy," Oct. 2018. Available at: https://arxiv.org/abs/1810.01835.

[13] J. Steinbaeck, C. Steger, G. Holweg, and N. Druml, "Next generation radar sensors in automotive sensor fusion systems," in *2017 Sensor Data Fusion: Trends, Solutions, Applications (SDF)*, Bonn, 2017, pp. 1-6.

[14] Q. Luo, Y. Cao, J. Liu, and A. Benslimane, "Localization and Navigation in Autonomous Driving: Threats and Countermeasures," in *IEEE Wireless Communications*, vol. 26, no. 4, pp. 38-45, Aug. 2019, doi: 10.1109/MWC.2019.1800533.

[15] S. Kumar and K. B. Moore, "The Evolution of Global Positioning System (GPS) Technology," *Journal of Science Education and Technology*, vol. 11, pp. 59-80, 2002, doi: 10.1023/A:1013999415003.

[16] Advanced Navigation, "Inertial Navigation System [Explained]," 2024. Available at: https://www.advancednavigation.com/tech-articles/inertial-navigation-systems-ins-an-introduction/ [Accessed Aug. 25, 2024].

[17] Advanced Navigation, "Inertial Measurement Unit (IMU) | An Introduction," 2024. Available at: https://www.advancednavigation.com/tech-articles/inertial-measurement-unit-imu-an-introduction/ [Accessed Aug. 25, 2024].

[18] K. Bimbraw, "Autonomous cars: Past, present and future - a review of the developments in the last century, the present scenario and the expected future of autonomous vehicle technology," in *2015 12th International Conference on Informatics in Control, Automation and Robotics (ICINCO)*, Colmar, France, 2015, pp. 191-198.

[19] M. Cranswick, *Pontiac Firebird*, United Kingdom: Veloce Publishing Limited, 2003.

[20] D. W. Temple, *GM's Motorama: The Glamorous Show Cars of a Cultural Phenomenon*, United States: MBI Publishing, 2006.
[21] M. Burgan, *The Pontiac Firebird*. United States: RiverFront Books, 1999.
[22] J. Pressnell, *Citroen DS: The Complete Story*. United Kingdom: Crowood Press UK, 1999.
[23] K. H. Cardew, "The automatic steering of vehicles: An experimental system fitted to a DS 19 Citroen car," 1970.
[24] M. Xie, L. Trassoudaine, J. Alizon, M. Thonnat, and J. Gallice, "Active and intelligent sensing of road obstacles: Application to the European Eureka-PROMETHEUS project," in *1993 (4th) International Conference on Computer Vision*, Berlin, Germany, 1993, pp. 616-623, doi: 10.1109/ICCV.1993.378154.
[25] M. G. Flyte and Loughborough University of Technology Department of Human Sciences, "The safe design of in-vehicle information and support systems: the human factors issues," *International Journal of Vehicle Design*, [Preprint], 1995.
[26] L. Davis et al., "Vision-based navigation: a status report," *Proc. Image Understanding Workshop*, vol. 1, pp. 153-169, 1987.
[27] J. W. Lowrie, M. Thomas, K. Gremban, and M. Turk, "The Autonomous Land Vehicle (ALV) Preliminary Road-Following Demonstration," *Proc. SPIE 0579, Intelligent Robots and Computer Vision IV*, Dec. 11, 1985, doi: 10.1117/12.950819.
[28] T. Kanade, S. Shafer, C. Thorpe, and M. Herbert, "Toward autonomous driving: the CMU Navlab. Part I - Perception," in *IEEE Intelligent Systems*, vol. 6, no. 4, pp. 31-42, 1991, doi: 10.1109/64.85919.
[29] D. A. Pomerleau, "Knowledge-Based Training of Artificial Neural Networks for Autonomous Robot Driving," in J. H. Connell and S. Mahadevan (Eds.), *Robot Learning*, The Springer International Series in Engineering and Computer Science, vol. 233, Springer, Boston, MA, 1993. doi: 10.1007/978-1-4615-3184-5_2.
[30] A. Broggi, M. Bertozzi, and A. Fascioli, "Architectural issues on vision-based automatic vehicle guidance: The experience of the ARGO project," *Real-Time Imaging*, vol. 6, no. 4, pp. 313-324, 2000, doi: 10.1006/rtim.1999.0191.
[31] S. E. Shladover et al., "Lane Assist Systems for Bus Rapid Transit, Volume I: Technology Assessment," UC Berkeley: California Partners for Advanced Transportation Technology, 2007. Retrieved from https://escholarship.org/uc/item/9df1w6z6.
[32] I. Andreasson, "Innovative Transit Systems: Survey of current developments," 2001.
[33] T. Hong, M. Abrams, T. Chang, and M. Shneier, "An Intelligent World Model for Autonomous Off-Road Driving," *Comput. Vis. Image Understanding J.*, 2001. doi: 10.1002/0471221619.ch1.
[34] P. Bellutta, R. Manduchi, L. Matthies, K. Owens, and A. Rankin, "Terrain perception for DEMO III," in *Proc. IEEE Intelligent Vehicles Symposium 2000*, Dearborn, MI, USA, 2000, pp. 326-331, doi: 10.1109/IVS.2000.898363.
[35] C. M. Shoemaker and J. A. Bornstein, "The Demo III UGV program: a testbed for autonomous navigation research," in *Proc. 1998 IEEE International Symposium on Intelligent Control (ISIC)* held jointly with IEEE International Symposium on Computational Intelligence in Robotics and Automation (CIRA), Gaithersburg, MD, USA, 1998, pp. 644-651, doi: 10.1109/ISIC.1998.713784.
[36] T. Hong, C. Rasmussen, T. Chang, and M. Shneier, "Road detection and tracking for autonomous mobile robots," *Proc. SPIE 4715, Unmanned Ground Vehicle Technology IV*, July 17, 2002, doi: 10.1117/12.474463.
[37] R. W. Wolcott and R. M. Eustice, "Robust LIDAR localization using multiresolution Gaussian mixture maps for autonomous driving," *The International Journal of Robotics Research*, vol. 36, no. 3, pp. 292-319, 2017, doi: 10.1177/0278364917696568.
[38] E. Takeuchi and T. Tsubouchi, "A 3-D Scan Matching using Improved 3-D Normal Distributions Transform for Mobile Robotic Mapping," in *2006 IEEE/RSJ International Conference on Intelligent Robots and Systems*, Beijing, China, 2006, pp. 3068-3073, doi: 10.1109/IROS.2006.282246.
[39] J.-H. Im, S.-H. Im, and G.-I. Jee, "Vertical Corner Feature Based Precise Vehicle Localization Using 3D LIDAR in Urban Area," *Sensors*, vol. 16, no. 8, p. 1268, 2016, doi: 10.3390/s16081268.
[40] J.-H. Im, S.-H. Im, and G.-I. Jee, "Extended Line Map-Based Precise Vehicle Localization Using 3D LIDAR," *Sensors*, vol. 18, no. 10, 3179, 2018, doi: 10.3390/s18103179.

[41] W. Burgard, O. Brock, and C. Stachniss, "Map-Based Precision Vehicle Localization in Urban Environments," in *Robotics: Science and Systems III*, MIT Press, 2008, pp. 121-128.

[42] J. Levinson and S. Thrun, "Robust vehicle localization in urban environments using probabilistic maps," in *2010 IEEE International Conference on Robotics and Automation*, Anchorage, AK, USA, 2010, pp. 4372-4378, doi: 10.1109/ROBOT.2010.5509700.

[43] G.-P. Gwon, W.-S. Hur, S.-W. Kim, and S.-W. Seo, "Generation of a Precise and Efficient Lane-Level Road Map for Intelligent Vehicle Systems," in *IEEE Transactions on Vehicular Technology*, vol. 66, no. 6, pp. 4517-4533, June 2017, doi: 10.1109/TVT.2016.2535210.

[44] M. Aldibaja, N. Suganuma, and K. Yoneda, "Improving localization accuracy for autonomous driving in snow-rain environments," in *2016 IEEE/SICE International Symposium on System Integration (SII)*, Sapporo, Japan, 2016, pp. 212-217, doi: 10.1109/SII.2016.7844000.

[45] M. Aldibaja, N. Suganuma, and K. Yoneda, "Robust Intensity-Based Localization Method for Autonomous Driving on Snow–Wet Road Surface," *IEEE Transactions on Industrial Informatics*, vol. 13, no. 5, 2017, doi: 10.1109/TII.2017.2713836.

[46] N. Akai, L. Y. Morales, E. Takeuchi, Y. Yoshihara, and Y. Ninomiya, "Robust localization using 3D NDT scan matching with experimentally determined uncertainty and road marker matching," in *2017 IEEE Intelligent Vehicles Symposium (IV)*, Los Angeles, CA, USA, 2017, pp. 1356-1363, doi: 10.1109/IVS.2017.7995900.

[47] L. Li, M. Yang, C. Wang, and B. Wang, "Hybrid Filtering Framework Based Robust Localization for Industrial Vehicles," in *IEEE Transactions on Industrial Informatics*, vol. 14, no. 3, pp. 941-950, March 2018, doi: 10.1109/TII.2017.2738016.

[48] D. Vivet, F. Gérossier, P. Checchin, L. Trassoudaine, and R. Chapuis, "Mobile Ground-Based Radar Sensor for Localization and Mapping: An Evaluation of two Approaches," *International Journal of Advanced Robotic Systems*, vol. 10, no. 8, 2013, doi: 10.5772/56636.

[49] E. Ward and J. Folkesson, "Vehicle localization with low cost radar sensors," in *2016 IEEE Intelligent Vehicles Symposium (IV)*, Gothenburg, Sweden, 2016, pp. 864-870, doi: 10.1109/IVS.2016.7535489.

[50] H. Ma, E. Smart, A. Ahmed, and D. Brown, "Radar Image-Based Positioning for USV Under GPS Denial Environment," in *IEEE Transactions on Intelligent Transportation Systems*, vol. 19, no. 1, pp. 72-80, Jan. 2018, doi: 10.1109/TITS.2017.2690577.

[51] M. Rapp, M. Hahn, M. Thom, J. Dickmann, and K. Dietmayer, "Semi-Markov process based localization using radar in dynamic environments," in *2015 IEEE 18th International Conference on Intelligent Transportation Systems*, Gran Canaria, Spain, 2015, pp. 423-429, doi: 10.1109/ITSC.2015.77.

[52] K. Yoneda, N. Hashimoto, R. Yanase, M. Aldibaja, and N. Suganuma, "Vehicle localization using 76GHz omnidirectional millimeter-wave radar for winter automated driving," in *2018 IEEE Intelligent Vehicles Symposium (IV)*, Changshu, China, 2018, pp. 971-977, doi: 10.1109/IVS.2018.8500378.

[53] F. Schuster, M. Wörner, C. G. Keller, M. Haueis, and C. Curio, "Robust localization based on radar signal clustering," in *2016 IEEE Intelligent Vehicles Symposium (IV)*, Gothenburg, Sweden, 2016, pp. 839-844, doi: 10.1109/IVS.2016.7535485.

[54] M. Rapp et al., "Probabilistic ego-motion estimation using multiple automotive radar sensors," *Robotics and Autonomous Systems*, vol. 89, pp. 136-146, 2017, doi: 10.1016/j.robot.2016.11.009.

[55] M. Rapp, M. Barjenbruch, K. Dietmayer, M. Hahn, and J. Dickmann, "A fast probabilistic ego-motion estimation framework for radar," in *2015 European Conference on Mobile Robots (ECMR)*, Lincoln, UK, 2015, pp. 1-6, doi: 10.1109/ECMR.2015.7324046.

[56] M. Cornick, J. Koechling, B. Stanley, and B. Zhang, "Localizing ground penetrating RADAR: A step toward robust autonomous ground vehicle localization," *J. Field Robotics*, vol. 33, pp. 82-102, 2016, doi: 10.1002/rob.21605.

[57] O. De Silva, G. K. I. Mann, and R. G. Gosine, "An ultrasonic and vision-based relative positioning sensor for multirobot localization," *IEEE Sensors Journal*, vol. 15, no. 3, pp. 1716-1726, March 2015, doi: 10.1109/JSEN.2014.2364684.

[58] J. I. Rejas et al., "Environment mapping using a 3D laser scanner for unmanned ground vehicles," *Microprocessors and Microsystems*, vol. 39, no. 8, pp. 939-949, 2015, doi: 10.1016/j.micpro.2015.10.003.

[59] M. Moussa, A. Moussa, and N. El-Sheimy, "Ultrasonic based heading estimation for aiding land vehicle navigation in GNSS denied environment," *Int. Arch. Photogramm. Remote Sens. Spatial Inf. Sci.*, vol. XLII-1, pp. 315-322, 2018, doi: 10.5194/isprs-archives-XLII-1-315-2018.

[60] S. Jung, J. Kim, and S. Kim, "Simultaneous localization and mapping of a wheel-based autonomous vehicle with ultrasonic sensors," *Artif Life Robotics*, vol. 14, pp. 186, 2009, doi: 10.1007/s10015-009-0650-9.

[61] J. Breßler, P. Reisdorf, M. Obst, and G. Wanielik, "GNSS positioning in non-line-of-sight context—A survey," in *2016 IEEE 19th International Conference on Intelligent Transportation Systems (ITSC)*, Rio de Janeiro, Brazil, 2016, pp. 1147-1154, doi: 10.1109/ITSC.2016.7795701.

[62] M. Schreiber, H. Königshof, A. -M. Hellmund, and C. Stiller, "Vehicle localization with tightly coupled GNSS and visual odometry," in *2016 IEEE Intelligent Vehicles Symposium (IV)*, Gothenburg, Sweden, 2016, pp. 858-863, doi: 10.1109/IVS.2016.7535488.

[63] S. Peyraud et al., "About non-line-of-sight satellite detection and exclusion in a 3D map-aided localization algorithm," *Sensors*, vol. 13, no. 1, pp. 829-847, 2013, doi: 10.3390/s130100829.

[64] A. Amini, R. M. Vaghefi, J. M. de la Garza, and R. M. Buehrer, "Improving GPS-based vehicle positioning for Intelligent Transportation Systems," in *2014 IEEE Intelligent Vehicles Symposium Proceedings*, Dearborn, MI, USA, 2014, pp. 1023-1029, doi: 10.1109/IVS.2014.6856592.

[65] R. M. Vaghefi, M. R. Gholami, R. M. Buehrer, and E. G. Strom, "Cooperative received signal strength-based sensor localization with unknown transmit powers," *IEEE Transactions on Signal Processing*, vol. 61, no. 6, pp. 1389-1403, March 15, 2013, doi: 10.1109/TSP.2012.2232664.

[66] S. G. Park and D. J. Cho, "Smart framework for GNSS-based navigation in urban environments," *Int. J. Satell. Commun. Network.*, vol. 35, pp. 123-137, 2017, doi: 10.1002/sat.1166.

[67] W. Lu, S. A. Rodríguez F., E. Seignez, and R. Reynaud, "Lane marking-based vehicle localization using low-cost GPS and open source map," *Unmanned Systems*, vol. 03, no. 04, pp. 239-251, 2015.

[68] M. Adjrad and P. D. Groves, "Enhancing least squares GNSS positioning with 3D mapping without accurate prior knowledge," *J Inst Navig*, vol. 64, pp. 75-91, 2017, doi: 10.1002/navi.178.

[69] J. Gim and C. Ahn, "IMU-based virtual road profile sensor for vehicle localization," *Sensors*, vol. 18, no. 10, 3344, 2018, doi: 10.3390/s18103344.

[70] W. Azree, M. Abdul Rahman, and H. Zamzuri, "Vehicle localization using wheel speed sensor (WSS) and inertial measurement unit (IMU)," *Journal of the Society of Automotive Engineers Malaysia*, vol. 2, no. 1, pp. 43-59, 2021, doi: 10.56381/jsaem.v2i1.76.

[71] Li, Xu, Jiang, Rong, Song, Xianghui, Li, Bin, "A tightly coupled positioning solution for land vehicles in urban canyons," *Journal of Sensors*, 5965716, 11 pages, 2017, doi: 10.1155/2017/5965716.

[72] K-W Chiang, T.T. Duong, and J-K Liao, "The performance analysis of a real-time integrated INS/GPS vehicle navigation system with abnormal GPS measurement elimination," *Sensors*, vol. 13, no. 8, 10599-10622, 2013, doi: 10.3390/s130810599.

[73] S. Wang, Z. Deng, and G. Yin, "An accurate GPS-IMU/DR data fusion method for driverless car based on a set of predictive models and grid constraints," *Sensors*, vol. 16, no. 3, 280, 2016, doi: 10.3390/s16030280.

[74] Z. Xiao, V. Havyarimana, T. Li, and D. Wang, "A nonlinear framework of delayed particle smoothing method for vehicle localization under non-Gaussian environment," *Sensors*, vol. 16, no. 5, 692, 2016, doi: 10.3390/s16050692.

[75] I. Belhajem, Y. ben Maissa, and A. Tamtaoui, "Improving low cost sensor based vehicle positioning with machine learning," *Control Engineering Practice*, vol. 74, pp. 168-176, 2018, doi: 10.1016/j.conengprac.2018.03.006.

[76] A. Ndjeng, D. Gruyer, and S. Glaser, "New likelihood updating for the IMM approach application to outdoor vehicles localization," in *2009 IEEE/RSJ International Conference on Intelligent Robots and Systems*, St. Louis, MO, USA, 2009, pp. 1223-1228, doi: 10.1109/IROS.2009.5353896.

[77] D. Gruyer and E. Pollard, "Credibilistic IMM likelihood updating applied to outdoor vehicle robust ego-localization," in *14th International Conference on Information Fusion*, Chicago, IL, USA, 2011, pp. 1-8.

[78] A. J. Dean, R. D. Martini, and S. N. Brennan, "Terrain-based Road vehicle localization using particle filters," in *2008 American Control Conference*, Seattle, WA, USA, 2008, pp. 236-241, doi: 10.1109/ACC.2008.4586497.

[79] E. I. Laftchiev, C. M. Lagoa, and S. N. Brennan, "Vehicle localization using in-vehicle pitch data and dynamical models," in *IEEE Transactions on Intelligent Transportation Systems*, vol. 16, no. 1, pp. 206-220, Feb. 2015, doi: 10.1109/TITS.2014.2330795.

[80] N. Piasco et al., "A survey on visual-based localization: On the benefit of heterogeneous data," *Pattern Recognition*, vol. 74, pp. 90-109, 2018, doi: 10.1016/j.patcog.2017.09.013.

[81] S. Sivaraman and M. M. Trivedi, "Looking at vehicles on the road: A survey of vision-based vehicle detection, tracking, and behavior analysis," in *IEEE Transactions on Intelligent Transportation Systems*, vol. 14, no. 4, pp. 1773-1795, Dec. 2013, doi: 10.1109/TITS.2013.2266661.

[82] S. Houben, M. Neuhausen, M. Michael, et al., "Park marking-based vehicle self-localization with a fisheye topview system," *J Real-Time Image Proc*, vol. 16, pp. 289-304, 2019, doi: 10.1007/s11554-015-0529-z.

[83] X. Du and K.K. Tan, "Vision-based approach towards lane line detection and vehicle localization," *Machine Vision and Applications*, vol. 27, pp. 175-191, 2016, doi: 10.1007/s00138-015-0735-5.

[84] Z. Xiao, K. Jiang, S. Xie, T. Wen, C. Yu, and D. Yang, "Monocular vehicle self-localization method based on compact semantic map," in *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, Maui, HI, USA, 2018, pp. 3083-3090, doi: 10.1109/ITSC.2018.8569274.

[85] K. Yoneda, R. Yanase, M. Aldibaja, N. Suganuma, and K. Sato, "Mono-camera based localization using digital map," in *IECON 2017 - 43rd Annual Conference of the IEEE Industrial Electronics Society*, Beijing, China, 2017, pp. 8285-8290, doi: 10.1109/IECON.2017.8217454.

[86] K. Yoneda, R. Yanase, M. Aldibaja, et al., "Mono-camera based vehicle localization using lidar intensity map for automated driving," *Artif Life Robotics*, vol. 24, pp. 147-154, 2019, doi: 10.1007/s10015-018-0502-6.

[87] T. Naseer, W. Burgard, and C. Stachniss, "Robust visual localization across seasons," in *IEEE Transactions on Robotics*, vol. 34, no. 2, pp. 289-302, April 2018, doi: 10.1109/TRO.2017.2788045.

[88] H.Y. Lin, C.W. Yao, K.S. Cheng, et al., "Topological map construction and scene recognition for vehicle localization," *Auton Robot*, vol. 42, pp. 65-81, 2018, doi: 10.1007/s10514-017-9638-9.

[89] S. Kato, E. Takeuchi, Y. Ishiguro, Y. Ninomiya, K. Takeda, and T. Hamada, "An open approach to autonomous vehicles," in *IEEE Micro*, vol. 35, no. 6, pp. 60-68, Nov.-Dec. 2015, doi: 10.1109/MM.2015.133.

[90] J.E. Deschaud, "KITTI-CARLA: a KITTI-like dataset generated by CARLA Simulator," arXiv preprint arXiv:2109.00892, 2021.

[91] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Conference on robot learning*, pp. 1-16, PMLR, 2017, October.

[92] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *2012 IEEE Conference on Computer Vision and Pattern Recognition*, Providence, RI, USA, 2012, pp. 3354-3361, doi: 10.1109/CVPR.2012.6248074.

[93] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, "Vision meets robotics: The KITTI dataset," *The International Journal of Robotics Research*, vol. 32, no. 11, pp. 1231-1237, 2013, doi: 10.1177/0278364913491297.

[94] F.L. Drake Jr. et al., "3.11.8 Documentation, Python," 2022. Available at: https://docs.python.org/3.11/ (Accessed: August 31, 2024).

Appendix A

Estimated Sensor Paths and Error Plots

Town 01

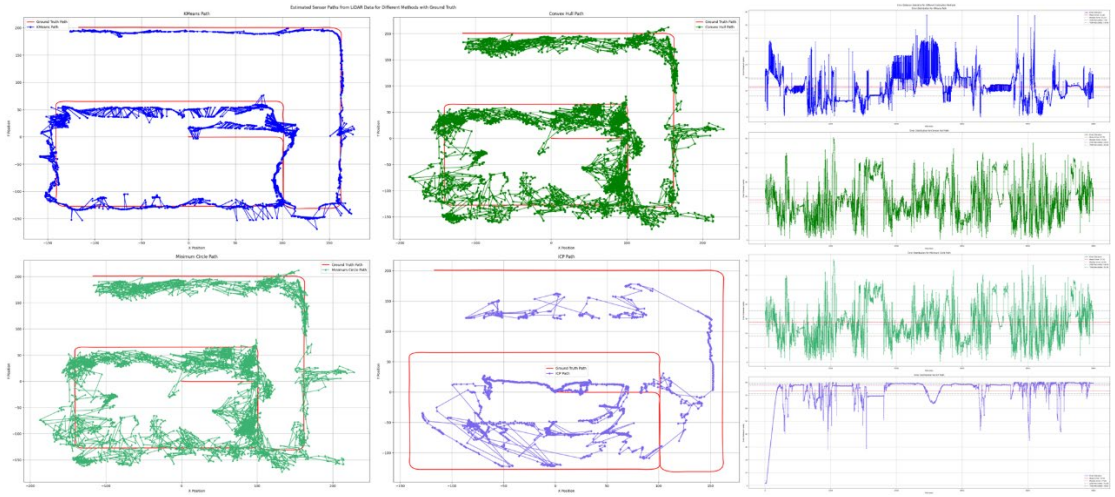

Figure 11: *Estimated Sensor Paths and Error Plots for Town 01*

Town 02

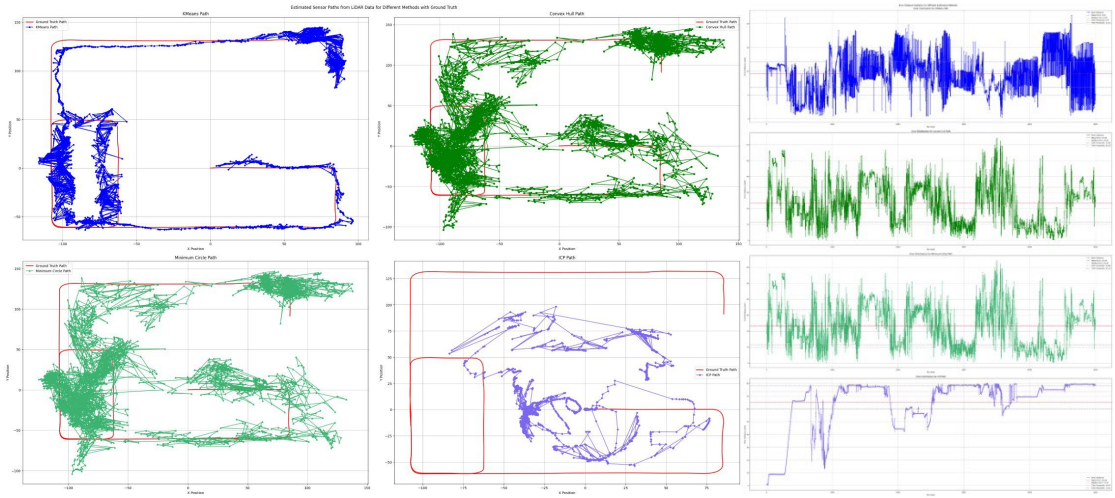

Figure 12: *Estimated Sensor Paths and Error Plots for Town 02*

Town 03

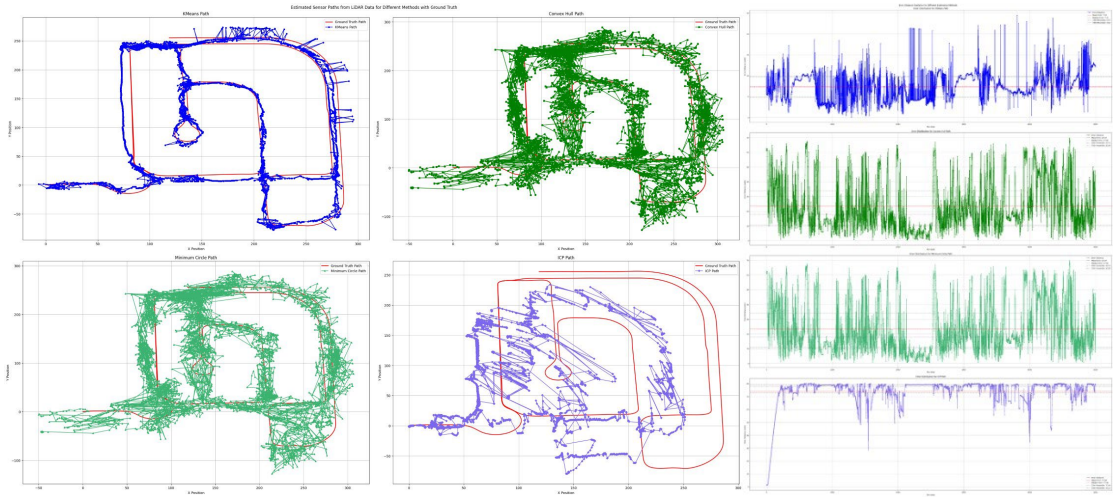

Figure 13: *Estimated Sensor Paths and Error Plots for Town 03*

Town 04

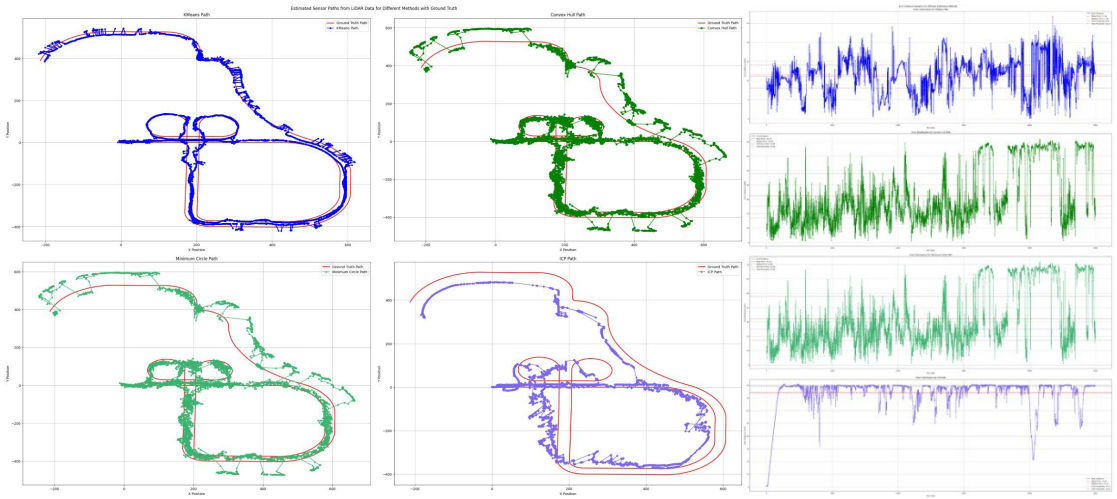

Figure 14: *Estimated Sensor Paths and Error Plots for Town 04*

Town 05

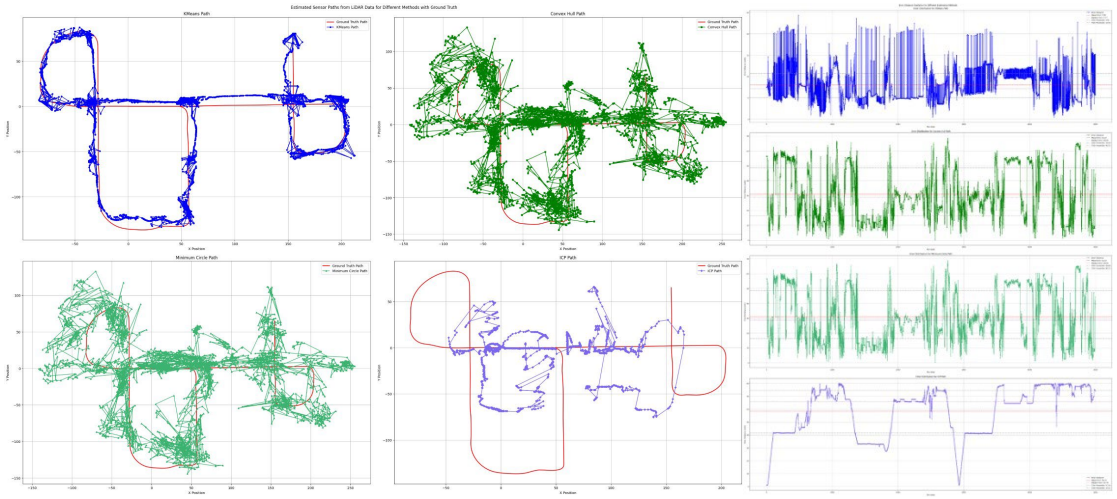

Figure 15: *Estimated Sensor Paths and Error Plots for Town 05*

Town 06

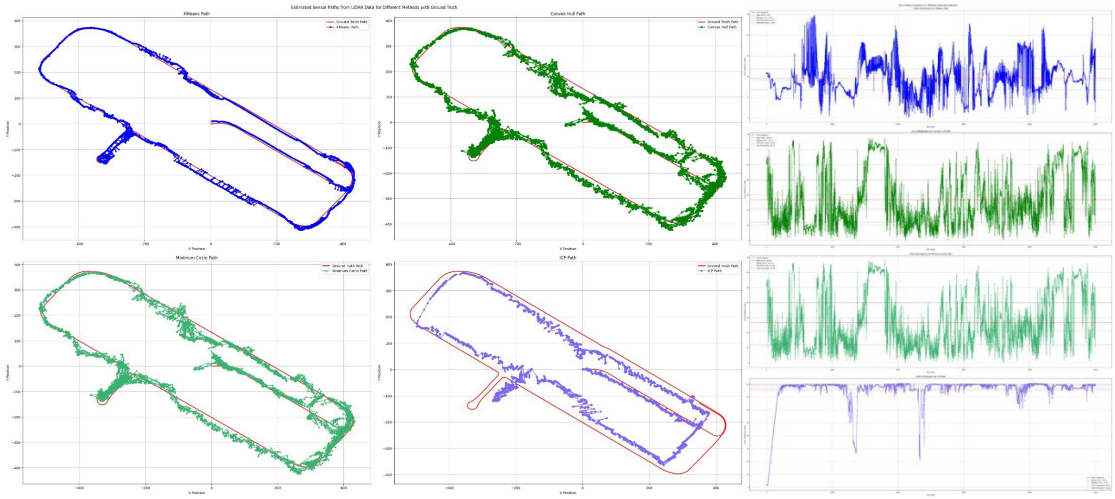

Figure 16: *Estimated Sensor Paths and Error Plots for Town 06*

Town 07

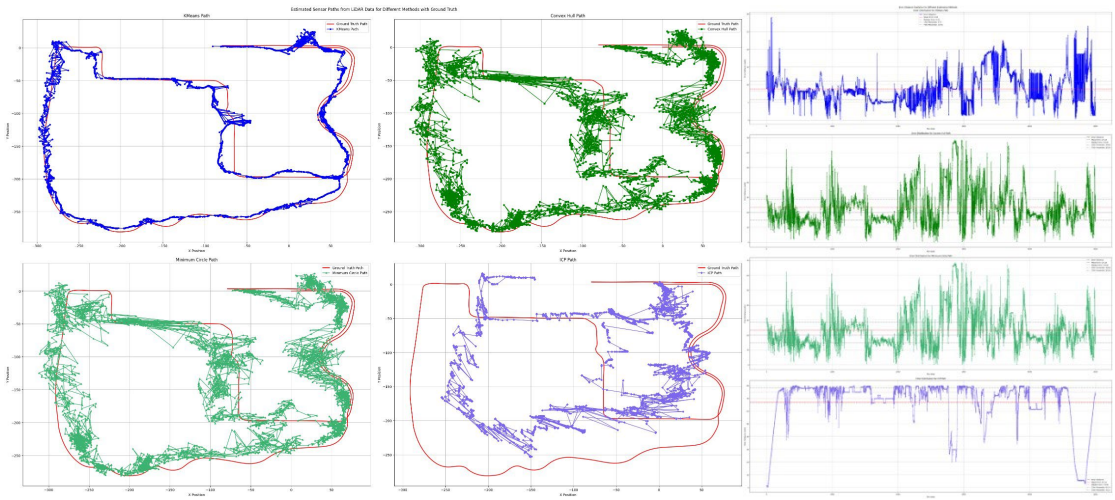

Figure 17: *Estimated Sensor Paths and Error Plots for Town 07*

Appendix B

Code and Dataset File Details

B.1 Dataset Description

The dataset used for this project can be found in the 03_Dataset on the link Dataset (https://bit.ly/datasetkvi1n23). The password for the dataset is datasetkvi1n23.

The dataset contains seven town folders. Each town folder contains the LiDAR data points which have already been converted to world coordinates. The LiDAR data points files are stored in the .ply file inside the frames_in_world folder. There are three other text files in each town folder. The file averaged_poses_lidar.txt contains the ground truth points. The other two files are the output files which are saved during the runtime of the code.

B.2 Code File

The code is written in the jupyter file. The Python version used during the project is Python 3.11 to guarantee maximum support for Python packages and libraries. Before running the jupyter file, ensure all the packages are installed in the system. It takes approximately 21 hours for the complete code to run.

B.3 Python Package List

The list of packages used in the code is as follows:

- os
- warnings
- time
- random
- logging
- numpy
- pandas
- seaborn
- tqdm
- scipy
- plyfile
- matplotlib
- sklearn